MATHEMATICAL FOUNDATIONS OF ARTIFICIAL INTELLIGENCE LECTURE NOTES

CÉSAR GALINDO

Abstract. These notes develop the mathematics behind modern deep learning, following the course *Mathematics of AI* taught by Eduardo Dueñez at the University of Texas at San Antonio in spring 2026, whose textbook is Goodfellow, Bengio and Courville's *Deep Learning*. The thread runs from linear algebra (the singular value decomposition and low-rank approximation), probability and information theory, through optimization and statistical learning (gradient descent, its stochastic versions, the bias–variance decomposition, double descent, support vector machines), to neural networks (the forward pass, backpropagation, convolutional, recurrent and attention architectures, memory in sequence models) and to generative and sequential models (variational autoencoders, autoregressive models, diffusion, reinforcement learning, geometric deep learning). Each section states its results precisely, proves them, and ends with exercises.

Contents

1. Learning from data: a first example 1
2. Linear algebra: tensors and the singular value decomposition 3
3. Probability 5
4. Information theory 7
5. Calculus and gradient-based optimization 9
6. Machine learning basics: capacity, bias, variance, double descent 11
7. Support vector machines 13
8. Feedforward networks: units and the forward pass 15
9. Backpropagation and automatic differentiation 17
10. Architectures of deep networks 19
11. Memory in sequence models 21
12. Generative models 23
13. Diffusion models and score-based generation 25
14. Reinforcement learning 27
15. Geometric deep learning 28
References 30

1. Learning from data: a first example

The main reference for these notes is [1]; see also [2, 3, 4].

Date: October 2026.

2020 *Mathematics Subject Classification.* 68T07, 62M45, 68T05, 15A18, 94A17, 90C15, 60J22.

Key words and phrases. deep learning, singular value decomposition, information theory, stochastic gradient descent, bias–variance, support vector machines, backpropagation, attention, variational autoencoders, diffusion models, reinforcement learning, equivariance.

1.1. **The supervised learning problem.** A *supervised learning task* consists of an input space $\mathcal{X}$, an output space $\mathcal{Y}$, an unknown probability distribution p_{data} on $\mathcal{X} \times \mathcal{Y}$, and a *loss* $\ell\colon \mathcal{Y} \times \mathcal{Y} \to [0, \infty)$. A *predictor* is a measurable map $h\colon \mathcal{X} \to \mathcal{Y}$, and its *risk* is

$$R(h) = \mathbb{E}_{(X,Y)\sim p_{\text{data}}}\big[\ell(Y, h(X))\big].$$

The distribution is not known; one only observes a *training set* $D = \{(x_i, y_i)\}_{i=1}^{m}$ of independent draws from p_{data}. A *model* is a family $\{h_\theta\}_{\theta\in\Theta}$, $\Theta \subseteq \mathbb{R}^P$, and *training* means approximately minimizing the *empirical risk*

$$\widehat{R}_D(\theta) = \frac{1}{m}\sum_{i=1}^{m} \ell(y_i, h_\theta(x_i)). \tag{1.1}$$

Performance is then measured on a *test set* drawn independently of D; a *validation set*, also disjoint from D, is used to choose quantities not learned by minimizing (1.1) (widths, depths, learning rates), which are called *hyperparameters*. The gap between $\widehat{R}_D$ and R is the subject of Section 6.

1.2. **Handwritten digits.** In the MNIST dataset, $\mathcal{X} = [0,1]^{784}$ (a 28×28 grayscale image flattened to a vector) and $\mathcal{Y} = \{0, 1, \dots, 9\}$; there are 60,000 training and 10,000 test images. A *feedforward network with two hidden layers* of widths n_1, n_2 is the map

$$h_\theta(x) = \operatorname{softmax}\Big(W^{(3)}\,\mathrm{ReLU}\big(W^{(2)}\,\mathrm{ReLU}(W^{(1)}x + b^{(1)}) + b^{(2)}\big) + b^{(3)}\Big) \in \Delta^9,$$

where $W^{(1)} \in \mathbb{R}^{n_1\times 784}$, $W^{(2)} \in \mathbb{R}^{n_2\times n_1}$, $W^{(3)} \in \mathbb{R}^{10\times n_2}$, the $b^{(l)}$ are vectors of matching size, $\mathrm{ReLU}(t) = \max(t, 0)$ acts entrywise, Δ^9 is the simplex of probability vectors in $\mathbb{R}^{10}$, and

$$\operatorname{softmax}(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}}.$$

The output is read as a probability distribution over the ten classes, the predicted class is $\arg\max_k h_\theta(x)_k$, and the loss is the *categorical cross-entropy* $\ell(y, p) = -\log p_y$. The entries of $z = W^{(3)}a^{(2)} + b^{(3)}$ are the *logits*. Training runs a stochastic version of gradient descent (Section 5) on (1.1), sweeping the training set repeatedly; one sweep is an *epoch*.

Proposition 1.1. *The network above has* $785n_1 + (n_1 + 1)n_2 + 10(n_2 + 1)$ *trainable parameters. For* $n_1 = n_2 = 64$ *this is* 55,050.

Proof. A layer from width n to width n' carries nn' weights and n' biases. Summing over the three layers gives the formula, and $50{,}240 + 4{,}160 + 650 = 55{,}050$. □

Even this small network reaches about 98% test accuracy after a few epochs. The rest of these notes explains, in order, the mathematics needed to understand why: the linear algebra of the maps $W^{(l)}$, the probability theory behind the softmax and the cross-entropy, the optimization that finds θ, the statistics that controls the gap between training and test error, and the architecture of the network itself.

Exercise 1.2. Count the parameters of the analogous networks with one hidden layer of width n_1 and with three hidden layers of widths n_1, n_2, n_3. For a fixed budget of 55,050 parameters, which widths are possible with one hidden layer?

Exercise 1.3. A `float32` number is stored as a sign bit, an 8-bit biased exponent e and a 23-bit fraction f, and represents $(-1)^s(1 + f2^{-23})2^{e-127}$ for $0 < e < 255$. Find the largest finite `float32`, the smallest positive normalized one, and the spacing between consecutive representable numbers in $[1, 2)$. Explain why $\operatorname{softmax}(z)$ should be computed as $\operatorname{softmax}(z - \max_j z_j)$, and prove that the two expressions agree.

2. Linear algebra: tensors and the singular value decomposition

2.1. **Vectors, matrices, tensors.** A *tensor* of order k and shape $(d_1, \dots, d_k)$ is an element of $\mathbb{R}^{d_1} \otimes \cdots \otimes \mathbb{R}^{d_k} \cong \mathbb{R}^{d_1 \times \cdots \times d_k}$, that is, an array $T = (T_{i_1 \dots i_k})$. Vectors and matrices are tensors of orders 1 and 2; a batch of B color images of size $H \times W$ is a tensor of shape $(B, H, W, 3)$. Two operations on arrays recur constantly. *Contraction* sums over a shared index, as in $(AB)_{ik} = \sum_j A_{ij} B_{jk}$; and *broadcasting* adds a vector $b \in \mathbb{R}^n$ to every row of a matrix $Z \in \mathbb{R}^{B \times n}$, i.e. forms $Z + \mathbf{1}_B b^\top$. A dense layer applied to a batch is the map $X \mapsto XW^\top + \mathbf{1} b^\top$.

We write $\langle x, y \rangle = x^\top y$, $\|x\|$ for the Euclidean norm, $\|A\| = \max_{\|x\|=1} \|Ax\|$ for the operator (spectral) norm, and $\|A\|_F = (\sum_{ij} A_{ij}^2)^{1/2} = (\operatorname{tr} A^\top A)^{1/2}$ for the Frobenius norm. A square matrix Q is *orthogonal* if $Q^\top Q = I$.

Theorem 2.1 (Spectral theorem)**.** *If $S \in \mathbb{R}^{n \times n}$ is symmetric, there is an orthogonal V and real $\lambda_1 \geq \cdots \geq \lambda_n$ with $S = V \operatorname{diag}(\lambda_1, \dots, \lambda_n) V^\top$. If moreover S is positive semidefinite, i.e. $x^\top S x \geq 0$ for all x, then all $\lambda_i \geq 0$.*

The proof is standard; positivity follows from $\lambda_i = v_i^\top S v_i$.

2.2. **The singular value decomposition.**

Theorem 2.2 (SVD)**.** *Let $A \in \mathbb{R}^{m \times n}$ have rank r. There are orthogonal $U \in \mathbb{R}^{m \times m}$, $V \in \mathbb{R}^{n \times n}$ and numbers $\sigma_1 \geq \cdots \geq \sigma_r > 0$ such that*

$$A = U \Sigma V^\top = \sum_{i=1}^{r} \sigma_i \, u_i v_i^\top,$$

where $\Sigma \in \mathbb{R}^{m \times n}$ has $\Sigma_{ii} = \sigma_i$ for $i \leq r$ and all other entries zero, and u_i, v_i are the columns of U, V. The singular values *σ_i are the square roots of the nonzero eigenvalues of $A^\top A$ (equivalently, of $AA^\top$).*

Proof. The matrix $A^\top A$ is symmetric and positive semidefinite, since $x^\top A^\top A x = \|Ax\|^2$. By Theorem 2.1 there is an orthonormal basis $v_1, \dots, v_n$ of $\mathbb{R}^n$ with $A^\top A v_i = \lambda_i v_i$ and $\lambda_1 \geq \cdots \geq \lambda_n \geq 0$. Since $A^\top A x = 0$ implies $\|Ax\|^2 = x^\top A^\top A x = 0$, we have $\ker A^\top A = \ker A$, so $A^\top A$ has rank r and exactly r of the λ_i are positive; moreover $\|Av_i\|^2 = \lambda_i$. Put $\sigma_i = \sqrt{\lambda_i}$ and $u_i = Av_i / \sigma_i$ for $i \leq r$. Then

$$\langle u_i, u_j \rangle = \frac{v_i^\top A^\top A v_j}{\sigma_i \sigma_j} = \frac{\lambda_j \langle v_i, v_j \rangle}{\sigma_i \sigma_j} = \delta_{ij},$$

so $u_1, \dots, u_r$ are orthonormal; extend them to an orthonormal basis $u_1, \dots, u_m$ of $\mathbb{R}^m$. By construction $Av_i = \sigma_i u_i$ for $i \leq r$ and $Av_i = 0$ for $i > r$, which says $AV = U\Sigma$. Multiplying on the right by $V^\top$ gives the claim. Finally $AA^\top u_i = A(\sigma_i v_i) = \sigma_i^2 u_i$ for $i \leq r$, so the nonzero eigenvalues of $AA^\top$ are the same σ_i^2. □

The v_i are the *right* and the u_i the *left singular vectors*. The singular values are determined by A; the singular vectors are not, as the next exercise shows.

Exercise 2.3. Let A be 3×2 of rank 2 with $\sigma_1 > \sigma_2$.

(a) Show that the pair (v_1, v_2) is determined up to the signs of v_1, v_2, that u_1, u_2 are then determined, and that u_3 is determined up to sign. Conclude that A has exactly 8 singular value decompositions.

(b) Show that a 4×2 matrix of rank 2 with $\sigma_1 > \sigma_2$ has infinitely many, parametrized by $\{\pm 1\}^2 \times O(2)$.

(c) Take $A = \left(\begin{smallmatrix} 1 & 2 \\ -1 & 1 \\ 2 & 1 \end{smallmatrix}\right)$. Compute its SVD exactly: the eigenvalues of $A^\top A$, an orthonormal eigenbasis, the u_i, and check $U^\top U = I$ and $A = \sigma_1 u_1 v_1^\top + \sigma_2 u_2 v_2^\top$.

2.3. **Low-rank approximation.** For $0 \le k \le r$ put $A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^{\top}$, a matrix of rank k.

Theorem 2.4 (Eckart–Young [5])**.** *For every $B \in \mathbb{R}^{m\times n}$ of rank at most k,*

$$\|A - B\| \ge \|A - A_k\| = \sigma_{k+1}, \qquad \|A - B\|_F^2 \ge \|A - A_k\|_F^2 = \sum_{i>k} \sigma_i^2,$$

with the convention $\sigma_i = 0$ for $i > r$.

Proof. The equalities follow from $A - A_k = \sum_{i>k} \sigma_i u_i v_i^{\top}$ and orthonormality. For the spectral norm, $\ker B$ has dimension at least $n-k$, so it meets the $(k+1)$-dimensional span of $v_1, \dots, v_{k+1}$ in a unit vector $x = \sum_{i\le k+1} c_i v_i$. Then

$$\|(A - B)x\|^2 = \|Ax\|^2 = \sum_{i\le k+1} \sigma_i^2 c_i^2 \ge \sigma_{k+1}^2.$$

We have shown $\sigma_{k+1}(M) = \min\{\|M - B\| : \operatorname{rank} B \le k\}$ for every matrix M. This gives Weyl's inequality $\sigma_{i+j-1}(X + Y) \le \sigma_i(X) + \sigma_j(Y)$: with X_{i-1}, Y_{j-1} the truncated decompositions, $X_{i-1} + Y_{j-1}$ has rank at most $i + j - 2$, so

$$\sigma_{i+j-1}(X + Y) \le \|X + Y - X_{i-1} - Y_{j-1}\| \le \sigma_i(X) + \sigma_j(Y).$$

Apply it with $X = A - B$, $Y = B$ and $j = k + 1$; since $\sigma_{k+1}(B) = 0$ we get $\sigma_{i+k}(A) \le \sigma_i(A - B)$ for all i, and summing squares, $\|A - B\|_F^2 = \sum_i \sigma_i(A - B)^2 \ge \sum_i \sigma_{i+k}(A)^2$. □

A grayscale image is a matrix of intensities, and A_k stores it with $k(m + n + 1)$ numbers instead of mn. For a 768×1024 photograph, $k = 10$ already gives a recognizable picture and a few dozen terms are visually indistinguishable from the original, because the singular values of natural images decay quickly.

2.4. **Principal component analysis.** Let $X \in \mathbb{R}^{N\times n}$ hold N centred data points $x_1, \dots, x_N$ as rows ($\sum_i x_i = 0$), and let $X = U\Sigma V^{\top}$. The *sample covariance* is $C = \frac{1}{N} X^{\top} X = V \frac{\Sigma^{\top}\Sigma}{N} V^{\top}$.

Proposition 2.5 (PCA)**.** *Among k-dimensional subspaces $W \subseteq \mathbb{R}^n$, the total squared distance $\sum_i \|x_i - P_W x_i\|^2$ from the data to W is minimized, and the variance $\sum_i \|P_W x_i\|^2$ of the projected data is maximized, by $W = \operatorname{span}(v_1, \dots, v_k)$. The minimum equals $\sum_{i>k} \sigma_i^2$.*

Proof. If $P_W = QQ^{\top}$ with $Q \in \mathbb{R}^{n\times k}$ having orthonormal columns, the projected data matrix is $XQQ^{\top}$, of rank at most k, and $\sum_i \|x_i - P_W x_i\|^2 = \|X - XQQ^{\top}\|_F^2 \ge \sum_{i>k} \sigma_i^2$ by Theorem 2.4. For $Q = (v_1 | \cdots | v_k)$ one gets $XQQ^{\top} = X_k$, so equality holds. The two problems are equivalent because $\|x_i\|^2 = \|P_W x_i\|^2 + \|x_i - P_W x_i\|^2$. □

The vectors v_i are the *principal directions*, $Xv_i = \sigma_i u_i$ are the *principal components*, and σ_i^2/N is the variance explained by the i-th component.

2.5. **The pseudoinverse and least squares.** The *Moore–Penrose pseudoinverse* of $A = \sum_{i\le r} \sigma_i u_i v_i^{\top}$ is $A^{+} = \sum_{i\le r} \sigma_i^{-1} v_i u_i^{\top}$.

Proposition 2.6. *For $y \in \mathbb{R}^m$, the vector $\hat{\beta} = A^{+} y$ minimizes $\|A\beta - y\|$, and among all minimizers it is the unique one of smallest norm. If A has full row rank, $A^{+} = A^{\top}(AA^{\top})^{-1}$ and $A\hat{\beta} = y$; if A has full column rank, $A^{+} = (A^{\top}A)^{-1}A^{\top}$.*

Proof. Write $\beta = \sum_j c_j v_j$. Then $\|A\beta - y\|^2 = \sum_{i\le r} (\sigma_i c_i - \langle u_i, y\rangle)^2 + \sum_{i>r} \langle u_i, y\rangle^2$, minimized exactly when $c_i = \langle u_i, y\rangle/\sigma_i$ for $i \le r$, the c_j with $j > r$ being free. The norm $\sum_j c_j^2$ is smallest when these free coefficients vanish, which is $\beta = A^{+} y$. The closed forms follow by substituting the SVD. □

Exercise 2.7. Let $A \in \mathbb{R}^{m\times n}$. Prove that $\|A\| = \sigma_1$, $\|A\|_F^2 = \sum_i \sigma_i^2$, and that the *condition number* $\kappa(A) = \sigma_1/\sigma_n$ of an invertible square matrix bounds the relative error amplification: if $Ax = y$ and $A(x + \delta x) = y + \delta y$, then $\|\delta x\|/\|x\| \le \kappa(A)\|\delta y\|/\|y\|$.

Exercise 2.8. Prove that the map $A \mapsto A_k$ is well defined (independent of the choice of SVD) whenever $\sigma_k > \sigma_{k+1}$, and give an example showing that it is not when $\sigma_k = \sigma_{k+1}$.

3. Probability

3.1. **Probability spaces and random vectors.** A *probability space* is a triple $(\Omega, \mathcal{F}, \mathbb{P})$: a set Ω, a σ-algebra $\mathcal{F}$ of subsets of Ω (closed under complements and countable unions, and containing Ω), and a countably additive $\mathbb{P}\colon \mathcal{F} \to [0,1]$ with $\mathbb{P}(\Omega) = 1$. A *random vector* is a measurable $X\colon \Omega \to \mathbb{R}^n$; its *distribution* is the probability measure $\mu_X(B) = \mathbb{P}(X \in B)$ on the Borel sets of $\mathbb{R}^n$. The components $X_1, \dots, X_n$ of a random vector are *jointly distributed* random variables, and μ_X is their joint distribution; the distribution of a sub-vector is a *marginal*.

The distribution is *discrete* if it is concentrated on a countable set, and is then described by a *probability mass function* $p(x) = \mathbb{P}(X = x)$. It is *absolutely continuous* if there is a *density* $p \ge 0$ with $\mathbb{P}(X \in B) = \int_B p(x)\,dx$. Marginal densities are obtained by integrating out the other variables. The *expectation* of $g(X)$ is $\mathbb{E}g(X) = \int g\,d\mu_X$, which is $\sum_x g(x)p(x)$ or $\int g(x)p(x)\,dx$ in the two cases. The *mean* of X is $\mu = \mathbb{E}X \in \mathbb{R}^n$ and its *covariance* is $\Sigma = \mathbb{E}[(X-\mu)(X-\mu)^\top]$, a symmetric positive semidefinite matrix since $a^\top \Sigma a = \operatorname{Var}(a^\top X) \ge 0$.

3.2. **Conditioning and independence.** For events with $\mathbb{P}(B) > 0$, $\mathbb{P}(A \mid B) = \mathbb{P}(A \cap B)/\mathbb{P}(B)$. Events A, B are *independent* if $\mathbb{P}(A \cap B) = \mathbb{P}(A)\mathbb{P}(B)$, and random vectors X, Y are independent if $\{X \in A\}$ and $\{Y \in B\}$ are independent for all Borel A, B; for densities this is $p_{X,Y}(x,y) = p_X(x)p_Y(y)$. When (X,Y) has a density, the *conditional density* of X given $Y = y$ is $p(x \mid y) = p(x,y)/p(y)$ wherever $p(y) > 0$.

Proposition 3.1 (Bayes' rule)**.** $p(x \mid y) = \dfrac{p(y \mid x)\,p(x)}{\int p(y \mid x')\,p(x')\,dx'}$, *and similarly with sums for mass functions.*

In Bayesian language, $p(x)$ is the *prior*, $p(y \mid x)$ the *likelihood*, the denominator the *evidence*, and $p(x \mid y)$ the *posterior*. The evidence integral is typically intractable; this is the origin of variational inference (Section 12).

Example 3.2. A test for a condition of prevalence 1% has sensitivity 99% and false-positive rate 5%. The probability of the condition given a positive test is $\frac{0.99\cdot 0.01}{0.99\cdot 0.01 + 0.05\cdot 0.99} = \frac{1}{6}$. Base rates dominate.

3.3. **Change of variables.**

Theorem 3.3. *Let X have density p_X supported on an open set $U \subseteq \mathbb{R}^n$, and let $f\colon U \to V$ be a C^1 diffeomorphism. Then $Y = f(X)$ has density*

$$p_Y(y) = p_X\big(f^{-1}(y)\big)\,\big|\det Df^{-1}(y)\big|, \qquad y \in V.$$

Proof. For a Borel set $B \subseteq V$, the change of variables formula for integrals gives $\mathbb{P}(Y \in B) = \int_{f^{-1}(B)} p_X(x)\,dx = \int_B p_X(f^{-1}(y))|\det Df^{-1}(y)|\,dy$. □

In particular, if $Y = AX + b$ with A invertible, $p_Y(y) = p_X(A^{-1}(y-b))/|\det A|$. This formula, applied to a learned invertible map, is the basis of normalizing flows (Section 12.6).

3.4. **The multivariate normal distribution.** The *standard normal* $\mathcal{N}(0, I_n)$ is the distribution of a vector Z of n independent $\mathcal{N}(0,1)$ variables, with density $(2\pi)^{-n/2}e^{-\|z\|^2/2}$.

Definition 3.4. A random vector $X \in \mathbb{R}^n$ is *(jointly) normal* with mean μ and covariance Σ, written $X \sim \mathcal{N}(\mu, \Sigma)$, if X has the distribution of $\mu + AZ$ for some $A \in \mathbb{R}^{n\times k}$ with $AA^\top = \Sigma$ and $Z \sim \mathcal{N}(0, I_k)$.

The distribution depends only on (μ, Σ), since its characteristic function is $\mathbb{E}e^{it^\top X} = e^{it^\top\mu - t^\top\Sigma t/2}$. Every symmetric positive semidefinite Σ occurs (take $A = \Sigma^{1/2}$), and affine images of normal vectors are normal: $BX + c \sim \mathcal{N}(B\mu + c, B\Sigma B^\top)$.

Proposition 3.5. *If Σ is positive definite, $\mathcal{N}(\mu, \Sigma)$ has density*

$$\mathcal{N}(x; \mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^n \det \Sigma}} \exp\Big(-\tfrac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)\Big).$$

Proof. Take $A = \Sigma^{1/2}$ in Theorem 3.3: $p_X(x) = (2\pi)^{-n/2}e^{-\|A^{-1}(x-\mu)\|^2/2}/\det A$, and $\|A^{-1}v\|^2 = v^\top\Sigma^{-1}v$, $\det A = \sqrt{\det\Sigma}$. □

Proposition 3.6 (Geometry of the level sets)**.** *Let $A \in \mathbb{R}^{n\times n}$ be invertible with SVD $A = U\,\mathrm{diag}(\sigma_i)V^\top$, and let $S = A^\top A$. The level sets of the density of $\mathcal{N}(0, S)$ are the ellipsoids*

$$E_c = \Big\{x : \tfrac{1}{2}x^\top S^{-1}x = c\Big\} = \Big\{x : \sum_{i=1}^{n} \frac{\langle v_i, x\rangle^2}{2c\,\sigma_i^2} = 1\Big\},$$

whose principal axes point along the right singular vectors v_i of A, with semi-axis lengths $\sigma_i\sqrt{2c}$. In particular, the ratios of the axes are the ratios of the singular values, and E_c is the image of the sphere of radius $\sqrt{2c}$ under the linear map $V\,\mathrm{diag}(\sigma_i)$, which is $A^\top$ up to an orthogonal change of coordinates.

Proof. $S = V\,\mathrm{diag}(\sigma_i^2)V^\top$, so $S^{-1} = V\,\mathrm{diag}(\sigma_i^{-2})V^\top$ and $x^\top S^{-1}x = \sum_i \langle v_i, x\rangle^2/\sigma_i^2$. In the orthonormal coordinates $y_i = \langle v_i, x\rangle$ the set E_c is $\sum_i y_i^2/(2c\sigma_i^2) = 1$. Finally $A^\top = V\,\mathrm{diag}(\sigma_i)U^\top$, and $U^\top$ preserves spheres. □

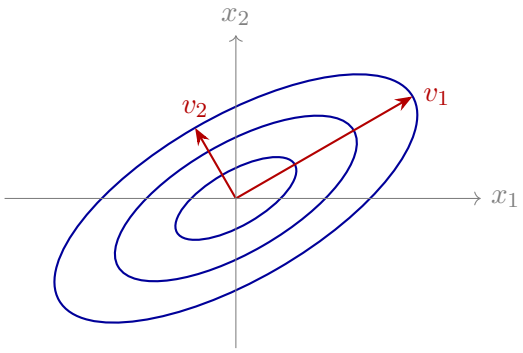


FIGURE 1. Level sets of $\mathcal{N}(0, A^\top A)$ for $n = 2$: ellipses with axes along v_1, v_2 and semi-axes proportional to $\sigma_1 > \sigma_2$.

If A is singular, then S has a nontrivial kernel, $\mathcal{N}(0, S)$ is concentrated on the proper subspace $\mathrm{im}\,S$, and there is no density on $\mathbb{R}^n$: the formula of Proposition 3.5 breaks down because $\det S = 0$ and S^{-1} does not exist. On $\mathrm{im}\,S$ itself the distribution is a non-degenerate normal.

Proposition 3.7 (Conditioning)**.** *Let* $\left(\begin{smallmatrix} X_1 \\ X_2 \end{smallmatrix}\right) \sim \mathcal{N}\Big(\left(\begin{smallmatrix} \mu_1 \\ \mu_2 \end{smallmatrix}\right), \left(\begin{smallmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{smallmatrix}\right)\Big)$ *with* Σ_{22} *invertible. Then*

$$X_1 \mid X_2 = x_2 \;\sim\; \mathcal{N}(\mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_2),\; \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}).$$

Proof. Let $W = X_1 - \Sigma_{12}\Sigma_{22}^{-1}X_2$. The pair (W, X_2) is an affine image of (X_1, X_2), hence jointly normal, and $\operatorname{Cov}(W, X_2) = \Sigma_{12} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{22} = 0$. Uncorrelated jointly normal vectors are independent (the characteristic function factors), so conditionally on $X_2 = x_2$, $X_1 = W + \Sigma_{12}\Sigma_{22}^{-1}x_2$ with $W \sim \mathcal{N}(\mu_1 - \Sigma_{12}\Sigma_{22}^{-1}\mu_2,\; \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21})$. □

Exercise 3.8. Show that if $X \sim \mathcal{N}(\mu, \Sigma)$ with Σ positive definite, then $(X-\mu)^\top\Sigma^{-1}(X-\mu)$ has the χ^2_n distribution. Deduce the probability that X lies inside the ellipsoid E_c of Proposition 3.6 for $n = 2$.

Exercise 3.9. Let $X \sim \mathcal{N}(0, \sigma^2)$ and $Y = e^X$. Use Theorem 3.3 to find the density of Y (the log-normal distribution) and compute $\mathbb{E}Y$ and $\operatorname{Var} Y$.

4. Information theory

4.1. **Entropy, cross-entropy, divergence.** Let p, q be probability mass functions on a countable set, or densities on $\mathbb{R}^n$; we write the discrete case, the continuous one being identical with integrals. Logarithms are natural (units: nats).

Definition 4.1. The *(Shannon) entropy* of p is $H(p) = -\sum_x p(x)\log p(x)$; the *cross-entropy* of q relative to p is $H(p, q) = -\sum_x p(x)\log q(x)$; and the *Kullback–Leibler divergence* is

$$D_{\mathrm{KL}}(p \,\|\, q) = \sum_x p(x)\log\frac{p(x)}{q(x)} = H(p, q) - H(p),$$

with the conventions $0\log 0 = 0$ and $D_{\mathrm{KL}}(p\|q) = +\infty$ if $p(x) > 0 = q(x)$ for some x. For densities, H is the *differential entropy*, which can be negative.

Entropy measures uncertainty: $-\log p(x)$ is the information gained by observing x, and $H(p)$ its average. By Shannon's source coding theorem, $H(p)/\log 2$ is the optimal average number of bits per symbol for encoding draws from p; encoding with a code optimal for q costs $H(p, q)/\log 2$ bits, and $D_{\mathrm{KL}}(p\|q)$ is the excess.

Theorem 4.2 (Gibbs' inequality)**.** $D_{\mathrm{KL}}(p\|q) \geq 0$*, with equality if and only if* $p = q$ *(*p*-almost everywhere in the continuous case).*

Proof. On the support of p, $-\log$ is strictly convex, so Jensen's inequality gives $D_{\mathrm{KL}}(p\|q) = \mathbb{E}_p\big[-\log\frac{q(X)}{p(X)}\big] \geq -\log\mathbb{E}_p\frac{q(X)}{p(X)} = -\log\sum_{p(x)>0} q(x) \geq -\log 1 = 0$. Equality in Jensen forces q/p to be constant on the support of p, and equality in the last step forces q to vanish off it; together, $p = q$. □

Corollary 4.3. *On a set with* K *elements,* $0 \leq H(p) \leq \log K$*, with* $H(p) = \log K$ *only for the uniform distribution* u*. Among densities on* $\mathbb{R}$ *with variance* σ^2*, the normal density maximizes differential entropy, which equals* $\frac{1}{2}\log(2\pi e\sigma^2)$*.*

Proof. $0 \leq D_{\mathrm{KL}}(p\|u) = \log K - H(p)$. For the second claim let φ be the $\mathcal{N}(\mu, \sigma^2)$ density with the same mean as p. Since $-\log\varphi(x) = \frac{1}{2}\log(2\pi\sigma^2) + (x-\mu)^2/(2\sigma^2)$ is quadratic, $H(p, \varphi) = H(\varphi, \varphi) = \frac{1}{2}\log(2\pi e\sigma^2)$, and $0 \leq D_{\mathrm{KL}}(p\|\varphi) = H(\varphi) - H(p)$. □

The divergence is not symmetric and does not satisfy the triangle inequality, so it is not a metric. Its asymmetry has a practical meaning: minimizing $D_{\mathrm{KL}}(p\|q)$ over q forces $q > 0$ wherever $p > 0$ ("mass-covering"), whereas minimizing $D_{\mathrm{KL}}(q\|p)$ forces $q = 0$ wherever $p = 0$ ("mode-seeking").

4.2. **Maximum likelihood is divergence minimization.** Let $\hat{p}_D = \frac{1}{m}\sum_i \delta_{x_i}$ be the empirical distribution of a sample and $\{q_\theta\}$ a model. Then

$$-\frac{1}{m}\sum_{i=1}^{m} \log q_\theta(x_i) = H(\hat{p}_D, q_\theta) = H(\hat{p}_D) + D_{\mathrm{KL}}(\hat{p}_D \| q_\theta),$$

so maximizing the likelihood, minimizing the cross-entropy and minimizing the divergence from the empirical distribution are the same problem. For a classifier $q_\theta(y \mid x)$, the categorical cross-entropy loss of Section 1 is the negative conditional log-likelihood.

4.3. **Closed forms.**

Proposition 4.4. *For univariate normals,*

$$D_{\mathrm{KL}}(\mathcal{N}(\mu_1, \sigma_1^2) \,\|\, \mathcal{N}(\mu_2, \sigma_2^2)) = \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}.$$

In particular $D_{\mathrm{KL}}(\mathcal{N}(0,1)\|\mathcal{N}(\mu, \sigma^2)) = \log \sigma + \frac{1+\mu^2}{2\sigma^2} - \frac{1}{2}$. *For n-variate normals with positive definite covariances,*

$$D_{\mathrm{KL}}(\mathcal{N}(\mu_1, \Sigma_1)\|\mathcal{N}(\mu_2, \Sigma_2)) = \frac{1}{2}\Big[\mathrm{tr}(\Sigma_2^{-1}\Sigma_1) + (\mu_2 - \mu_1)^\top \Sigma_2^{-1}(\mu_2 - \mu_1) - n + \log \frac{\det \Sigma_2}{\det \Sigma_1}\Big].$$

Proof. With $X \sim \mathcal{N}(\mu_1, \Sigma_1)$,

$$\log \frac{p_1(X)}{p_2(X)} = \frac{1}{2}\log \frac{\det \Sigma_2}{\det \Sigma_1} - \frac{1}{2}(X - \mu_1)^\top \Sigma_1^{-1}(X - \mu_1) + \frac{1}{2}(X - \mu_2)^\top \Sigma_2^{-1}(X - \mu_2).$$

Take expectations using $\mathbb{E}[(X-a)^\top M (X-a)] = \mathrm{tr}(M\Sigma_1) + (\mu_1 - a)^\top M(\mu_1 - a)$: the middle term gives $-n/2$ and the last gives $\frac{1}{2}[\mathrm{tr}(\Sigma_2^{-1}\Sigma_1) + (\mu_1 - \mu_2)^\top \Sigma_2^{-1}(\mu_1 - \mu_2)]$. The univariate case is $n = 1$. □

Let B_p denote the Bernoulli distribution, $B_p(1) = p$, $B_p(0) = 1 - p$.

Proposition 4.5. *For $p, q \in (0,1)$, $D_{\mathrm{KL}}(B_p\|B_q) = p\log\frac{p}{q} + (1-p)\log\frac{1-p}{1-q}$, and*

$$D_{\mathrm{KL}}(B_p\|B_{1-p}) = D_{\mathrm{KL}}(B_{1-p}\|B_p) = (2p-1)\log\frac{p}{1-p}.$$

Proof. The first formula is the definition. Substituting $q = 1-p$ gives $p\log\frac{p}{1-p} + (1-p)\log\frac{1-p}{p} = (2p-1)\log\frac{p}{1-p}$, which is invariant under $p \mapsto 1-p$. Conceptually: the relabelling $\tau\colon 0 \leftrightarrow 1$ is a bijection, divergence is invariant under applying the same bijection to both arguments, and $\tau_* B_p = B_{1-p}$; hence $D_{\mathrm{KL}}(B_p\|B_{1-p}) = D_{\mathrm{KL}}(\tau_* B_p \| \tau_* B_{1-p}) = D_{\mathrm{KL}}(B_{1-p}\|B_p)$. □

Exercise 4.6. Describe the regions of the unit square where $D_{\mathrm{KL}}(B_p\|B_q)$ is smaller than, equal to, or larger than $D_{\mathrm{KL}}(B_q\|B_p)$. Show that the equality set contains the diagonal $p = q$ and the anti-diagonal $p + q = 1$, and investigate numerically whether it contains anything else.

Exercise 4.7. The *mutual information* of (X, Y) is $I(X;Y) = D_{\mathrm{KL}}(p_{X,Y}\|p_X p_Y)$. Prove $I(X;Y) = H(X) - H(X \mid Y) \geq 0$, with equality if and only if X, Y are independent, and compute I for a bivariate normal with correlation ρ.

5. Calculus and gradient-based optimization

5.1. **Derivatives.** A map $f\colon \mathbb{R}^n \to \mathbb{R}^m$ is differentiable at x if there is a linear map $Df(x) \in \mathbb{R}^{m\times n}$, the *Jacobian*, with $f(x+h) = f(x) + Df(x)h + o(\|h\|)$. For $m = 1$ the *gradient* is $\nabla f(x) = Df(x)^\top$, the direction of steepest ascent: by Cauchy–Schwarz, among unit vectors u the directional derivative $\langle \nabla f(x), u\rangle$ is largest for $u = \nabla f(x)/\|\nabla f(x)\|$. The *chain rule* $D(g\circ f)(x) = Dg(f(x))\,Df(x)$ is the single fact behind backpropagation (Section 9). The *Hessian* $\nabla^2 f(x)$ is the symmetric matrix of second partials, and Taylor's theorem gives $f(x+h) = f(x) + \langle \nabla f(x), h\rangle + \frac{1}{2}h^\top \nabla^2 f(x) h + o(\|h\|^2)$ for $f \in C^2$.

A point with $\nabla f(x) = 0$ is *critical*. If $\nabla^2 f(x)$ is positive definite it is a strict local minimum; if $\nabla^2 f(x)$ has eigenvalues of both signs it is a *saddle*.

5.2. **Gradient descent.** To minimize $f\colon \mathbb{R}^n \to \mathbb{R}$, *gradient descent* with *learning rate* $\eta > 0$ iterates

$$w_{t+1} = w_t - \eta \nabla f(w_t).$$

We say f is L-*smooth* if ∇f is L-Lipschitz; for $f \in C^2$ this means $\|\nabla^2 f\| \le L$ everywhere.

Lemma 5.1 (Descent lemma)**.** *If f is L-smooth, then $f(y) \le f(x) + \langle \nabla f(x), y-x\rangle + \frac{L}{2}\|y-x\|^2$ for all x, y.*

Proof. $f(y) - f(x) - \langle \nabla f(x), y-x\rangle = \int_0^1 \langle \nabla f(x+s(y-x)) - \nabla f(x), y-x\rangle\, ds \le \int_0^1 Ls\|y-x\|^2 ds$. □

Theorem 5.2. *Let f be L-smooth and bounded below by f^*, and $0 < \eta < 2/L$. Then $f(w_{t+1}) \le f(w_t) - \eta(1 - \frac{L\eta}{2})\|\nabla f(w_t)\|^2$, and*

$$\min_{0\le t<T} \|\nabla f(w_t)\|^2 \le \frac{f(w_0) - f^*}{\eta(1 - L\eta/2)\,T}.$$

If in addition f is μ-strongly convex ($\nabla^2 f \succeq \mu I$) with minimizer w^* and $\eta \le 1/L$, then $f(w_t) - f(w^*) \le (1-\eta\mu)^t (f(w_0) - f(w^*))$.

Proof. Apply Lemma 5.1 with $x = w_t$, $y = w_t - \eta\nabla f(w_t)$ to get the first inequality, and sum it over $t < T$. For the second part, strong convexity gives $f(w^*) \ge f(w) + \langle \nabla f(w), w^* - w\rangle + \frac{\mu}{2}\|w^* - w\|^2 \ge f(w) - \frac{1}{2\mu}\|\nabla f(w)\|^2$ (minimize the right side over w^*), i.e. $\|\nabla f(w)\|^2 \ge 2\mu(f(w) - f(w^*))$. With $\eta \le 1/L$ the first inequality gives $f(w_{t+1}) - f(w^*) \le f(w_t) - f(w^*) - \frac{\eta}{2}\|\nabla f(w_t)\|^2 \le (1-\eta\mu)(f(w_t) - f(w^*))$. □

The threshold $2/L$ is sharp, as quadratics show.

Proposition 5.3. *Let $f(w) = \frac{1}{2}w^\top H w$ with H symmetric positive definite, eigenvalues $\lambda_1 \ge \cdots \ge \lambda_n > 0$. Gradient descent converges to 0 from every starting point if and only if $\eta < 2/\lambda_1$. The fastest uniform rate is attained at $\eta = 2/(\lambda_1 + \lambda_n)$, where $\|w_t\| \le \left(\frac{\kappa-1}{\kappa+1}\right)^t \|w_0\|$ with $\kappa = \lambda_1/\lambda_n$.*

Proof. $w_{t+1} = (I - \eta H)w_t$, and in an eigenbasis of H each coordinate is multiplied by $1 - \eta\lambda_i$ per step. Convergence for all w_0 means $|1 - \eta\lambda_i| < 1$ for all i, i.e. $\eta < 2/\lambda_1$. The rate is $\max_i |1 - \eta\lambda_i| = \max(|1 - \eta\lambda_1|, |1 - \eta\lambda_n|)$, minimized when the two are equal. □

Above the threshold the iterates oscillate with growing amplitude along the top eigendirection: the learning rate must be matched to the largest curvature, while the speed is governed by the smallest. Ill-conditioning ($\kappa \gg 1$) produces the familiar zig-zag down a narrow valley.

Proposition 5.4 (Saddles are unstable)**.** *For $f(w_1, w_2) = w_1^2 - w_2^2$ and $0 < \eta < 1$, gradient descent from (a, b) gives $w_t = \big((1-2\eta)^t a, (1+2\eta)^t b\big)$. It converges to the saddle $(0,0)$ only if $b = 0$; otherwise $|w_{2,t}| \to \infty$.*

The set of initial points attracted to a strict saddle has measure zero in general, but iterates started near the stable manifold linger there for a time logarithmic in the distance to it, which is why saddles slow training down without trapping it.

5.3. **Stochastic gradient descent.** In learning, $f(w)=\frac{1}{m}\sum_{i=1}^{m} f_i(w)$ with $f_i(w)=\ell(y_i,h_w(x_i))$, and evaluating ∇f costs a pass over all m examples. *Stochastic gradient descent* (SGD) replaces it by the gradient on a random *mini-batch* $\mathcal{B}_t\subseteq\{1,\dots,m\}$ of size B:

$$g_t=\frac{1}{B}\sum_{i\in\mathcal{B}_t}\nabla f_i(w_t),\qquad w_{t+1}=w_t-\eta_t g_t.$$

If $\mathcal{B}_t$ is drawn uniformly, $\mathbb{E}[g_t\mid w_t]=\nabla f(w_t)$: the step is unbiased, and with replacement its covariance is $\frac{1}{B}$ times the per-example gradient covariance. One step costs B/m of a full gradient. The noise has two effects: it slows convergence near the minimum (constant η only reaches a neighbourhood of size proportional to η), and it helps escape saddles and sharp minima.

Theorem 5.5. *Let F be μ-strongly convex with minimizer w^*, and let g_t satisfy $\mathbb{E}[g_t\mid w_1,\dots,w_t]=\nabla F(w_t)$ and $\mathbb{E}\|g_t\|^2\le G^2$. With $\eta_t=\frac{1}{\mu(t+1)}$,*

$$\mathbb{E}\|w_t-w^*\|^2\le\frac{Q}{t},\qquad Q=\max\Big(\|w_1-w^*\|^2,\frac{G^2}{\mu^2}\Big).$$

Proof. Let $a_t=\mathbb{E}\|w_t-w^*\|^2$. Expanding, $\|w_{t+1}-w^*\|^2=\|w_t-w^*\|^2-2\eta_t\langle g_t,w_t-w^*\rangle+\eta_t^2\|g_t\|^2$. Conditioning on the past and using strong convexity together with $\nabla F(w^*)=0$, $\langle\nabla F(w),w-w^*\rangle\ge\mu\|w-w^*\|^2$, we get $a_{t+1}\le(1-2\mu\eta_t)a_t+\eta_t^2G^2=\frac{t-1}{t+1}a_t+\frac{G^2}{\mu^2(t+1)^2}$. The claim holds for $t=1$. If $a_t\le Q/t$, then since $\frac{t-1}{t+1}\ge 0$,

$$a_{t+1}\le Q\Big(\frac{t-1}{t(t+1)}+\frac{1}{(t+1)^2}\Big)\le Q\Big(\frac{t-1}{t(t+1)}+\frac{1}{t(t+1)}\Big)=\frac{Q}{t+1}.\qquad\square$$

The step sizes satisfy the Robbins–Monro conditions [8] $\sum\eta_t=\infty$, $\sum\eta_t^2<\infty$: the first lets the iterates travel any distance, the second makes the accumulated noise $\sum\eta_t^2\|g_t\|^2$ summable. In practice one uses a constant rate followed by a decay schedule, for instance the cosine schedule $\eta_t=\frac{\eta_0}{2}(1+\cos(\pi t/T))$.

5.4. **Momentum and adaptive methods.** *Heavy-ball momentum* keeps a velocity: $v_{t+1}=\beta v_t+g_t$, $w_{t+1}=w_t-\eta v_{t+1}$ with $\beta\in[0,1)$. Unrolled, $v_{t+1}=\sum_{s\le t}\beta^{t-s}g_s$ is an exponentially weighted sum of past gradients, so components that keep their sign accumulate (up to a factor $1/(1-\beta)$) while oscillating ones cancel. *Nesterov momentum* evaluates the gradient at the look-ahead point $w_t-\eta\beta v_t$; on smooth convex functions it attains the optimal rate $O(1/t^2)$ of first-order methods, against $O(1/t)$ for gradient descent.

Adam [9] keeps exponential averages of the gradient and of its entrywise square,

$$m_t=\beta_1 m_{t-1}+(1-\beta_1)g_t,\qquad s_t=\beta_2 s_{t-1}+(1-\beta_2)g_t\odot g_t,$$

with $m_0=s_0=0$, forms $\hat m_t=m_t/(1-\beta_1^t)$, $\hat s_t=s_t/(1-\beta_2^t)$, and steps $w_{t+1}=w_t-\eta\,\hat m_t/(\sqrt{\hat s_t}+\varepsilon)$ entrywise. Each coordinate thus gets its own effective learning rate, of size about η regardless of the scale of its gradient. RMSProp is the same without m_t.

Proposition 5.6 (Bias correction)**.** *If $\mathbb{E}g_s=g$ for all s, then $\mathbb{E}m_t=(1-\beta_1^t)g$, so $\hat m_t$ is unbiased; likewise for $\hat s_t$ when $\mathbb{E}[g_s\odot g_s]$ is constant.*

Proof. $m_t=(1-\beta_1)\sum_{s=1}^{t}\beta_1^{t-s}g_s$, so $\mathbb{E}m_t=(1-\beta_1)\frac{1-\beta_1^t}{1-\beta_1}g$. $\square$

Exercise 5.7. For $f(w) = w^4/4 - w^2$ started at $w_0 = 2$, find the largest constant learning rate for which gradient descent converges to the minimizer $\sqrt{2}$, in terms of $f''(\sqrt{2})$. What happens just above it?

Exercise 5.8. Analyse heavy-ball momentum on $f(w) = \frac{\lambda}{2}w^2$: write the iteration as a linear recurrence on (w_t, w_{t-1}), find the eigenvalues of its matrix, and determine for which (η, β) it converges. Show that with the optimal choice for H as in Proposition 5.3 the rate improves from $\frac{\kappa-1}{\kappa+1}$ to $\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$.

Exercise 5.9. On $f(w_1, w_2) = w_1^2 - w_2^2$ started at $(0.01, 0.01)$, compare gradient descent, momentum and Adam. Show that Adam's first steps along w_2 have size close to η irrespective of how small $\partial f/\partial w_2$ is, and explain why this lets it leave the saddle faster.

6. Machine learning basics: capacity, bias, variance, double descent

6.1. **Tasks and capacity.** *Supervised* tasks learn a map $x \mapsto y$ from labelled pairs: *classification* ($\mathcal{Y}$ finite) and *regression* ($\mathcal{Y} = \mathbb{R}^k$). *Unsupervised* tasks learn structure of $p_{\text{data}}(x)$ from unlabelled data: density estimation, clustering, dimensionality reduction, generation. The *capacity* of a model is, informally, the variety of functions it can fit. A model *underfits* when its training error is large, and *overfits* when the gap between test and training error is large.

6.2. **No free lunch.**

Theorem 6.1 (No free lunch [7])**.** *Let $\mathcal{X}$ be finite, $\mathcal{Y} = \{0, 1\}$, and let $S \subset \mathcal{X}$ be a set of training inputs. Let A be any learning algorithm, which given the labelled sample $\{(x, f(x)) : x \in S\}$ outputs a predictor $h = A(f|_S)$. If the target f is drawn uniformly from all $2^{|\mathcal{X}|}$ functions $\mathcal{X} \to \{0, 1\}$, then for every $x \notin S$,*

$$\mathbb{P}(A(f|_S)(x) \neq f(x)) = \tfrac{1}{2}.$$

Consequently all algorithms have the same expected off-training-set error.

Proof. Under the uniform distribution the values $f(x)$, $x \in \mathcal{X}$, are independent fair bits. For $x \notin S$, the prediction $A(f|_S)(x)$ is a function of $f|_S$ (and of independent internal randomness), which is independent of $f(x)$; hence it equals $f(x)$ with probability $\frac{1}{2}$. □

Learning is therefore only possible relative to assumptions about which targets are likely; these assumptions, built into the choice of model and training procedure, are the *inductive bias*. Convolution (Section 10) and equivariance (Section 15) are inductive biases.

6.3. **The bias–variance decomposition.** Let $y = f(x) + \varepsilon$ with $\mathbb{E}\varepsilon = 0$, $\operatorname{Var}\varepsilon = \sigma^2$, the noise at the test point being independent of the training set D. A learning procedure produces $\hat{f}_D$; write $\bar{f}(x) = \mathbb{E}_D \hat{f}_D(x)$.

Theorem 6.2. *For every fixed x,*

$$\mathbb{E}_{D,\varepsilon}\big[(y - \hat{f}_D(x))^2\big] = \underbrace{\sigma^2}_{\text{noise}} + \underbrace{\big(f(x) - \bar{f}(x)\big)^2}_{\text{bias}^2} + \underbrace{\mathbb{E}_D\big[(\hat{f}_D(x) - \bar{f}(x))^2\big]}_{\text{variance}}.$$

Proof. Write $y - \hat{f}_D = \varepsilon + (f - \bar{f}) + (\bar{f} - \hat{f}_D)$. Square and take expectations. The cross terms vanish: $\mathbb{E}[\varepsilon(\cdot)] = 0$ because ε is centred and independent of D; $\mathbb{E}_D[(f - \bar{f})(\bar{f} - \hat{f}_D)] = (f - \bar{f})\mathbb{E}_D[\bar{f} - \hat{f}_D] = 0$ because $f - \bar{f}$ does not depend on D. □

Integrating over x gives the same decomposition of the expected test error. Low-capacity models have large bias and small variance; high-capacity ones the reverse. When capacity is tuned along a single parameter (degree of a polynomial, number of features), the test error traces the *classical U-curve*.

Example 6.3. Fit polynomials of degree d by least squares to $n = 20$ noisy samples of $\sin x$ on $[0, 2\pi]$. Degrees 0 and 1 have large bias; degree 3 to 5 fit well; near $d = 19$ the fitted curves pass through every point and oscillate wildly between them, and the variance term explodes.

6.4. **The interpolation threshold.** For features $\phi(x) = (\phi_1(x), \dots, \phi_p(x))$, linear least squares minimizes $\|\Phi\beta - y\|^2$, where $\Phi \in \mathbb{R}^{n\times p}$ has rows $\phi(x_i)^\top$. For polynomials, $\Phi = V$ is the Vandermonde matrix $V_{ij} = x_i^{j-1}$, $p = d + 1$.

Proposition 6.4. *If $x_1, \dots, x_n$ are distinct and $d \geq n - 1$, polynomial least squares of degree d interpolates: the training error is zero.*

Proof. The first n columns of V form the square Vandermonde matrix, with determinant $\prod_{i<j}(x_j - x_i) \neq 0$. So V has rank n, $V\beta = y$ is solvable, and its solutions have zero residual. □

For $p \geq n$ and Φ of full row rank there are infinitely many interpolants, and the natural choice is the *minimum-norm interpolant* $\hat\beta = \Phi^+ y = \Phi^\top(\Phi\Phi^\top)^{-1}y$ of Proposition 2.6, which is also the limit of gradient descent on $\|\Phi\beta - y\|^2$ started at $\beta = 0$ (the iterates never leave the row space of Φ). Its norm is

$$\|\hat\beta\|^2 = y^\top(\Phi\Phi^\top)^{-1}y = \sum_{i=1}^{n} \frac{\langle u_i, y\rangle^2}{\sigma_i^2},$$

where σ_i, u_i come from the SVD of Φ. At the *interpolation threshold* $p \approx n$ the matrix Φ is square and typically nearly singular: σ_n is close to 0, the coefficients blow up, and so does the variance of the fit. For random features, this is quantified by random matrix theory: if the entries of Φ are i.i.d. $\mathcal{N}(0, 1/p)$ and $n/p \to \gamma \in (0, 1)$, the eigenvalues of $\Phi\Phi^\top$ follow the Marchenko–Pastur law, supported on $[(1 - \sqrt{\gamma})^2, (1 + \sqrt{\gamma})^2]$, whose lower edge reaches 0 exactly as $\gamma \to 1$.

6.5. **Double descent.** Past the threshold, adding more features makes the problem better conditioned: σ_n moves away from zero and the minimum-norm interpolant becomes smoother, so the test error decreases again. The resulting *double descent curve* [6] has a classical U-shaped regime for $p < n$, a peak at $p \approx n$, and a second descent for $p > n$, often to a lower error than the bottom of the U. The same shape appears as a function of width, of training time (*epoch-wise double descent*) and, for deep networks, in the phenomenon of *grokking*, where test accuracy jumps long after training accuracy has saturated.

6.6. **Regularization.** *Ridge regression* minimizes $\|\Phi\beta - y\|^2 + \lambda\|\beta\|^2$, with solution $\hat\beta_\lambda = (\Phi^\top\Phi + \lambda I)^{-1}\Phi^\top y$.

Proposition 6.5. *For $\lambda > 0$, $\hat\beta_\lambda = \sum_i \frac{\sigma_i}{\sigma_i^2+\lambda}\langle u_i, y\rangle v_i$ exists even when $\Phi^\top\Phi$ is singular, satisfies $\|\hat\beta_\lambda\| \leq \|y\|/(2\sqrt{\lambda})$, and $\hat\beta_\lambda \to \Phi^+ y$ as $\lambda \to 0^+$.*

Proof. $\Phi^\top\Phi + \lambda I$ has eigenvalues $\sigma_i^2 + \lambda \geq \lambda > 0$ (including $\sigma_i = 0$), so it is invertible, and the formula follows from the SVD. Since $\sigma^2 + \lambda \geq 2\sigma\sqrt{\lambda}$, each coefficient satisfies $\frac{\sigma}{\sigma^2+\lambda} \leq \frac{1}{2\sqrt{\lambda}}$, and $\sum_i \langle u_i, y\rangle^2 \leq \|y\|^2$. As $\lambda \to 0$ each coefficient tends to $1/\sigma_i$ for $\sigma_i > 0$. □

The factor $\sigma_i^2/(\sigma_i^2 + \lambda)$ shrinks precisely the directions with small singular value, which are the ones responsible for the peak; with a well-chosen λ the double descent peak disappears. Ridge regression is also the maximum a posteriori estimator under a Gaussian prior $\beta \sim \mathcal{N}(0, \frac{\sigma^2}{\lambda} I)$.

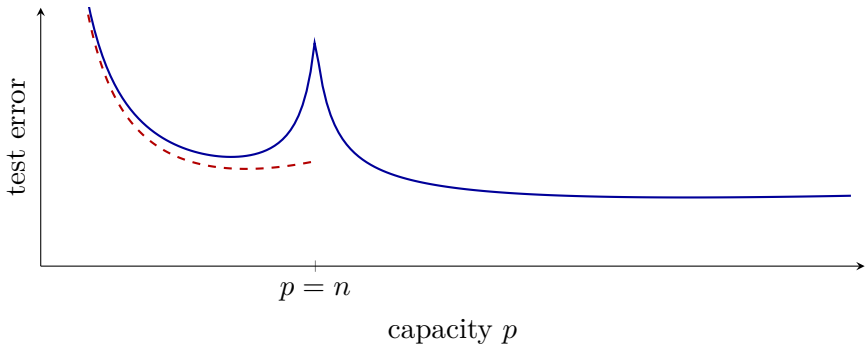


FIGURE 2. Classical U-curve (dashed) and double descent (solid): the test error peaks at the interpolation threshold $p = n$ and decreases again in the over-parametrized regime.

Other regularizers used for networks are weight decay (the same penalty), early stopping, dropout and data augmentation.

Exercise 6.6. Prove Theorem 6.2 for the expected test error averaged over $x \sim p(x)$, and show that for ridge regression with fixed design the bias increases and the variance decreases monotonically in λ.

Exercise 6.7. Show that the condition number of the $n \times (d+1)$ Vandermonde matrix on n equally spaced points of $[-1, 1]$ grows exponentially in d, and relate the growth near $d = n - 1$ to the peak of the double descent curve.

Exercise 6.8. Let $\Phi = [\phi_1 | \cdots | \phi_p] \in \mathbb{R}^{n \times p}$ with columns $\phi_j \sim \mathcal{N}(0, I_n/p)$ i.i.d., so that $\Phi\Phi^\top$ is Wishart. Using $\mathbb{E}[(\Phi\Phi^\top)^{-1}] = \frac{p}{p-n-1} I_n$ for $p > n+1$, show that for fixed y the minimum-norm interpolant satisfies $\mathbb{E}\|\hat\beta\|^2 = \frac{p}{p-n-1}\|y\|^2$, finite for $p > n+1$ and diverging as $p \to (n+1)^+$.

7. Support vector machines

7.1. **Maximum margin.** Let $(x_i, y_i)_{i=1}^m$ with $x_i \in \mathbb{R}^n$ and $y_i \in \{\pm 1\}$. A hyperplane $H_{w,b} = \{x : \langle w, x\rangle + b = 0\}$ *separates* the data if $y_i(\langle w, x_i\rangle + b) > 0$ for all i. The distance from x to $H_{w,b}$ is $|\langle w, x\rangle + b|/\|w\|$, and the *margin* of a separating hyperplane is $\min_i y_i(\langle w, x_i\rangle + b)/\|w\|$. Rescaling (w, b) so that $\min_i y_i(\langle w, x_i\rangle + b) = 1$, the margin is $1/\|w\|$ and the slab between $\langle w, x\rangle + b = \pm 1$ has width $2/\|w\|$. The *hard-margin SVM* [10] is the convex quadratic program

$$\min_{w,b} \ \tfrac{1}{2}\|w\|^2 \quad \text{subject to} \quad y_i(\langle w, x_i\rangle + b) \ge 1, \quad i = 1, \dots, m. \tag{7.1}$$

7.2. **Duality.** The Lagrangian of (7.1) is $\mathcal{L}(w, b, \alpha) = \tfrac{1}{2}\|w\|^2 - \sum_i \alpha_i\big(y_i(\langle w, x_i\rangle + b) - 1\big)$, $\alpha \ge 0$. Setting the derivatives in w and b to zero gives

$$w = \sum_i \alpha_i y_i x_i, \qquad \sum_i \alpha_i y_i = 0,$$

and substituting yields the *dual problem*

$$\max_{\alpha \ge 0} \ \sum_i \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j\rangle \quad \text{subject to} \quad \sum_i \alpha_i y_i = 0. \tag{7.2}$$

Theorem 7.1. *If the data are linearly separable, (7.1) has a unique solution w^*, strong duality holds, and for any dual solution α^*: $w^* = \sum_i \alpha_i^* y_i x_i$, and $\alpha_i^* > 0$ only if $y_i(\langle w^*, x_i\rangle + b^*) = 1$*

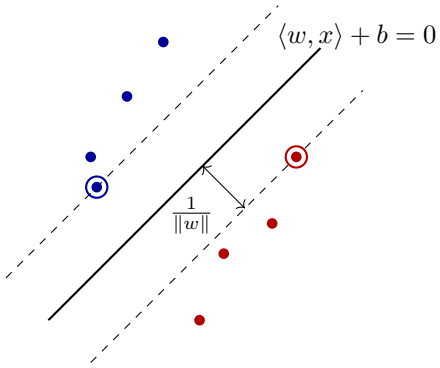


FIGURE 3. The maximum-margin hyperplane; the circled points are support vectors.

(complementary slackness). The x_i with $\alpha_i^* > 0$ are the support vectors*; the classifier depends only on them.*

Proof. The feasible set is a nonempty closed convex polyhedron and $\frac{1}{2}\|w\|^2$ is strictly convex and coercive in w, so the optimal w^* exists and is unique. For a convex problem with affine constraints, feasibility suffices for strong duality and for the Karush–Kuhn–Tucker conditions to characterize optimality (the refined Slater condition). Stationarity gives the formula for w^*, and complementary slackness $\alpha_i^*(y_i(\langle w^*, x_i\rangle + b^*) - 1) = 0$ gives the second claim. □

The bias is recovered from any support vector: $b^* = y_i - \langle w^*, x_i\rangle$. The dual involves the data only through the inner products $\langle x_i, x_j\rangle$, and so does the decision function $\operatorname{sign}(\sum_i \alpha_i^* y_i \langle x_i, x\rangle + b^*)$.

7.3. **Soft margin.** For non-separable data one introduces slack variables:

$$\min_{w,b,\xi} \ \tfrac{1}{2}\|w\|^2 + C\sum_i \xi_i \quad \text{subject to} \quad y_i(\langle w, x_i\rangle + b) \geq 1 - \xi_i,\ \xi_i \geq 0.$$

At the optimum $\xi_i = \max(0, 1 - y_i f(x_i))$ with $f(x) = \langle w, x\rangle + b$, so the problem is the unconstrained minimization of $\frac{1}{2}\|w\|^2 + C\sum_i \max(0, 1 - y_i f(x_i))$: *hinge loss* plus an ℓ^2 penalty, an instance of regularized empirical risk minimization. The dual is (7.2) with the box constraint $0 \leq \alpha_i \leq C$. Large C penalizes violations heavily (low bias, high variance); small C favours a wide margin.

7.4. **Kernels.** Replacing x by a feature vector $\varphi(x)$ in a Hilbert space $\mathcal{H}$ changes only the inner products, $\langle x_i, x_j\rangle \mapsto k(x_i, x_j) = \langle \varphi(x_i), \varphi(x_j)\rangle_{\mathcal{H}}$. This is the *kernel trick*: one never computes φ. For example, on $\mathbb{R}^2$, $k(x, z) = \langle x, z\rangle^2$ corresponds to $\varphi(x) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$, under which two concentric circles become linearly separable.

Definition 7.2. A symmetric function $k\colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is a *positive definite kernel* if every Gram matrix $(k(x_i, x_j))_{i,j\leq N}$ is positive semidefinite.

Theorem 7.3 (Moore–Aronszajn [11])**.** *k is a positive definite kernel if and only if $k(x, z) = \langle \varphi(x), \varphi(z)\rangle_{\mathcal{H}}$ for some Hilbert space $\mathcal{H}$ and map φ. One may take $\mathcal{H} = \mathcal{H}_k$, the unique Hilbert space of functions on $\mathcal{X}$ in which $k(\cdot, x) \in \mathcal{H}_k$ and $f(x) = \langle f, k(\cdot, x)\rangle$ for all f (the* reproducing kernel Hilbert space*), and $\varphi(x) = k(\cdot, x)$.*

Proposition 7.4. *Positive definite kernels are closed under sums, positive scalar multiples, products and pointwise limits. Consequently polynomial kernels $(\langle x, z\rangle + c)^d$ with $c \geq 0$, the exponential kernel $e^{\gamma\langle x,z\rangle}$ and the Gaussian (RBF) kernel $e^{-\gamma\|x-z\|^2}$, $\gamma > 0$, are positive definite.*

Proof. Sums, scalings and limits preserve positive semidefiniteness of Gram matrices. For products, the Gram matrix of $k_1 k_2$ is the entrywise (Hadamard) product $K_1 \circ K_2$; writing $K_2 = \sum_\ell \lambda_\ell u_\ell u_\ell^\top$ with $\lambda_\ell \geq 0$, we get $a^\top (K_1 \circ K_2) a = \sum_\ell \lambda_\ell (a \circ u_\ell)^\top K_1 (a \circ u_\ell) \geq 0$ (Schur's product theorem). The linear kernel and constants $c \geq 0$ are positive definite, hence so are $(\langle x, z\rangle + c)^d$ and, as a limit of partial sums of the exponential series with nonnegative coefficients, $e^{\gamma\langle x,z\rangle}$. Finally $e^{-\gamma\|x-z\|^2} = g(x)\, e^{2\gamma\langle x,z\rangle}\, g(z)$ with $g(x) = e^{-\gamma\|x\|^2}$, and $a^\top (DKD) a = (Da)^\top K (Da)$ for $D = \operatorname{diag}(g(x_i))$. □

The RBF feature space is infinite-dimensional, and γ sets its resolution: large γ makes $k(x, z)$ negligible unless x is very close to z, giving highly flexible boundaries that overfit; small γ gives nearly linear ones.

Theorem 7.5 (Representer theorem)**.** *Let $L\colon \mathbb{R}^m \to \mathbb{R}$ be arbitrary and $\Omega\colon [0,\infty) \to \mathbb{R}$ strictly increasing. Every minimizer of $L(f(x_1), \dots, f(x_m)) + \Omega(\|f\|_{\mathcal{H}_k})$ over $f \in \mathcal{H}_k$ has the form $f = \sum_{i=1}^m c_i k(\cdot, x_i)$.*

Proof. Decompose $f = f_\parallel + f_\perp$ with $f_\parallel \in \operatorname{span}\{k(\cdot, x_i)\}$ and $f_\perp$ orthogonal to it. By the reproducing property $f(x_i) = \langle f, k(\cdot, x_i)\rangle = f_\parallel(x_i)$, so the loss term depends only on $f_\parallel$, while $\|f\|^2 = \|f_\parallel\|^2 + \|f_\perp\|^2$. A minimizer must therefore have $f_\perp = 0$. □

The soft-margin SVM with kernel k is the case of hinge loss and $\Omega(t) = t^2/2$, and its solution $\sum_i \alpha_i y_i k(\cdot, x_i)$ agrees with the theorem. Multi-class problems are reduced to binary ones, one-vs-rest (K classifiers) or one-vs-one ($\binom{K}{2}$ classifiers with voting).

Exercise 7.6. Solve the hard-margin problem by hand for the four points $(1,1), (2,2)$ labelled $+1$ and $(0,0), (1,0)$ labelled -1: find the support vectors, w^*, b^*, the margin, and check that the dual optimum equals the primal one.

Exercise 7.7. Show that $k(x, z) = \min(x, z)$ on $[0,\infty)$ is a positive definite kernel by exhibiting a feature map into $L^2[0,\infty)$. Show that $k(x, z) = \|x - z\|$ is not positive definite on $\mathbb{R}$.

8. Feedforward networks: units and the forward pass

8.1. **Architecture.** A *feedforward network* with architecture $(n_0, n_1, \dots, n_L)$ is the composite $h_\theta = F_L \circ \cdots \circ F_1$ of layers $F_l\colon \mathbb{R}^{n_{l-1}} \to \mathbb{R}^{n_l}$,

$$\mathbf{a}^{(0)} = \mathbf{x}, \qquad \mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \qquad \mathbf{a}^{(l)} = \phi_l(\mathbf{z}^{(l)}),$$

with weights $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$, biases $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$, and *activation functions* ϕ_l applied entrywise (except for the output activation, which may act on the whole vector). The i-th coordinate of layer l is a *unit*: it computes an affine function $\langle w_i, \mathbf{a}^{(l-1)}\rangle + b_i$ of its inputs, the *pre-activation*, and passes it through ϕ_l. Layers $1, \dots, L-1$ are *hidden*; L is the *depth*, $\max_l n_l$ the *width*.

Proposition 8.1. *The network has $\sum_{l=1}^{L} n_l (n_{l-1} + 1)$ parameters. Widening both hidden layers of $(784, 64, 64, 10)$ to 128 raises the count from $55{,}050$ to $118{,}282$, a factor of about 2.15.*

8.2. **Why nonlinearity.**

Proposition 8.2. *If every ϕ_l is affine, h_θ is affine: $h_\theta(\mathbf{x}) = W\mathbf{x} + \mathbf{b}$ with $W = W^{(L)} \cdots W^{(1)}$ (when $\phi_l = \mathrm{id}$), and $\operatorname{rank} W \leq \min_l n_l$. Depth adds no expressive power, and narrow layers strictly reduce it.*

Proof. Compositions of affine maps are affine; the rank bound holds because W factors through $\mathbb{R}^{n_l}$ for each l. □

Nonlinear activations break this collapse.

Theorem 8.3 (Universal approximation [12, 13])**.** *Let $\phi\colon \mathbb{R} \to \mathbb{R}$ be continuous. The functions $\mathbf{x} \mapsto \sum_{j=1}^{N} c_j \phi(\langle w_j, \mathbf{x}\rangle + b_j)$, $N \in \mathbb{N}$, are dense in $C(K)$ for every compact $K \subset \mathbb{R}^n$ in the uniform norm, if and only if ϕ is not a polynomial.*

For the ReLU in one variable there is a short constructive proof.

Proposition 8.4. *Every continuous piecewise linear $g\colon [a,b] \to \mathbb{R}$ with breakpoints $a < t_1 < \cdots < t_k < b$ equals $c_0 + c_1(x-a) + \sum_{j=1}^{k} d_j \operatorname{ReLU}(x - t_j)$ on $[a,b]$, where c_1 is the initial slope and d_j the change of slope at t_j. Hence one-hidden-layer ReLU networks are dense in $C[a,b]$.*

Proof. Both sides are continuous, piecewise linear with the same breakpoints, and have the same value at a and the same slope on each piece. Density follows because piecewise linear interpolants of a continuous function on finer and finer grids converge uniformly (uniform continuity), and $x - a = \operatorname{ReLU}(x-a)$ on $[a,b]$. □

Universality is an existence statement about width; it says nothing about how many units are needed or whether training finds them. Depth helps with both: there are functions computed by deep narrow ReLU networks that require exponentially many units with one hidden layer (for instance, the k-fold composition of the tent map has 2^k linear pieces and is computed with $O(k)$ units).

Example 8.5 (XOR)**.** No affine classifier separates $\{(0,0),(1,1)\}$ (label 0) from $\{(1,0),(0,1)\}$ (label 1): if $\langle w, x\rangle + b$ were positive on the second pair and negative on the first, adding gives $w_1 + w_2 + 2b > 0$ from one pair and < 0 from the other. Two hidden ReLU units suffice:

$$h(x) = \operatorname{ReLU}(x_1 + x_2) - 2\operatorname{ReLU}(x_1 + x_2 - 1)$$

takes the values $0, 1, 1, 0$ at $(0,0),(1,0),(0,1),(1,1)$. One hidden unit does not: with a single unit $u = \operatorname{ReLU}(\langle w, x\rangle + b)$, any thresholded output $\{c\,u + d > 0\}$ is a superlevel or sublevel set of $\langle w, x\rangle$, i.e. a half-plane, the empty set or the whole plane.

8.3. **Activation functions.** The common hidden activations are

$$\sigma(t) = \frac{1}{1+e^{-t}}, \qquad \tanh t = 2\sigma(2t) - 1, \qquad \operatorname{ReLU}(t) = \max(0,t),$$
$$\text{leaky ReLU}_\alpha(t) = \max(\alpha t, t), \qquad \operatorname{GELU}(t) = t\,\Phi(t),$$

with Φ the standard normal distribution function. Their derivatives govern how gradients flow backwards (Section 9): $\sigma' = \sigma(1-\sigma) \le \frac{1}{4}$ and $\tanh' = 1 - \tanh^2 \le 1$ decay to 0 for large $|t|$ (*saturation*), whereas ReLU′ is 1 on the positive half-line and 0 on the negative one (a unit whose pre-activation is negative on all inputs is *dead*).

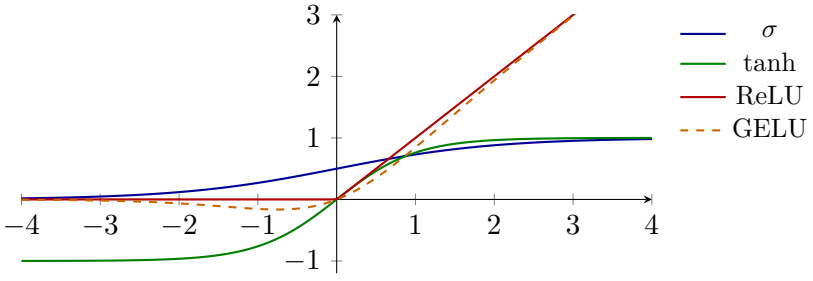


FIGURE 4. Hidden-layer activations.

8.4. **Output units and likelihoods.** The output activation is dictated by the probabilistic model of y given $\mathbf{x}$, and the loss is the corresponding negative log-likelihood (Section 4):

- *Regression*: identity output, $y \mid \mathbf{x} \sim \mathcal{N}(h(\mathbf{x}), \sigma^2 I)$, loss $\frac{1}{2\sigma^2}\|y - h(\mathbf{x})\|^2$ up to constants: mean squared error.
- *Multi-label detection* (K attributes, any subset present): entrywise sigmoid, independent Bernoulli outputs, loss the binary cross-entropy $-\sum_k [y_k \log \hat{y}_k + (1-y_k)\log(1-\hat{y}_k)]$.
- *Single-label classification* (K classes): softmax output, categorical distribution, loss $-\log \hat{y}_y$.

Proposition 8.6. *Let $p = \mathrm{softmax}(z)$. Then $\mathrm{softmax}(z + c\mathbf{1}) = \mathrm{softmax}(z)$ for every $c \in \mathbb{R}$,*

$$D\,\mathrm{softmax}(z) = \mathrm{diag}(p) - pp^\top,$$

and the cross-entropy $\ell(z) = -\log p_y$ has gradient $\nabla_z \ell = p - e_y$.

Proof. Shift invariance: the factor e^c cancels. Differentiating $p_i = e^{z_i}/\sum_j e^{z_j}$ gives $\partial p_i/\partial z_k = p_i\delta_{ik} - p_i p_k$. Finally $\ell = -z_y + \log\sum_j e^{z_j}$, so $\partial\ell/\partial z_k = -\delta_{yk} + p_k$. □

The softmax with *temperature* $\tau > 0$ is $\mathrm{softmax}(z/\tau)$; it tends to the uniform distribution as $\tau \to \infty$ and to the indicator of $\arg\max z$ as $\tau \to 0^+$ (when the maximum is unique). The sigmoid is the softmax of the two logits $(z, 0)$.

Exercise 8.7. Show that a network with $L = 5$ linear layers $\phi_l(t) = \alpha t$ computes an affine map, compute it, and find the rank of its linear part in terms of the widths.

Exercise 8.8. Derive the derivatives σ', $\tanh'$, GELU', and show that $\mathrm{GELU}(t) = \mathbb{E}[t\,\mathbf{1}_{\{Z \le t\}}]$ for $Z \sim \mathcal{N}(0,1)$, a "smoothed" ReLU in which the input is kept with a probability increasing in its size.

9. Backpropagation and automatic differentiation

9.1. **Computational graphs.** A *computational graph* is a finite directed acyclic graph whose source nodes are inputs and parameters and whose every other node v computes from its parents $u_1, \dots, u_k$ the value $x_v = f_v(x_{u_1}, \dots, x_{u_k})$ by an elementary differentiable operation. The *forward pass* evaluates the nodes in a topological order. Let ℓ be a scalar output node. The *adjoint* of a node is $\bar{x}_v = \partial\ell/\partial x_v$, the derivative of ℓ with respect to x_v along all paths from v to ℓ.

Proposition 9.1 (Reverse accumulation)**.** *$\bar{x}_\ell = 1$, and for every other node*

$$\bar{x}_u = \sum_{v:\, u \to v} \bar{x}_v \,\frac{\partial f_v}{\partial x_u},$$

the sum running over the children v of u. Processing the nodes in reverse topological order therefore computes all adjoints, in particular the gradient of ℓ with respect to every parameter, in one backward sweep.

Proof. This is the multivariate chain rule: ℓ depends on x_u only through the children of u, and by induction on the reverse topological order the $\bar{x}_v$ are already correct when $\bar{x}_u$ is formed. □

In words: *adjoint = upstream gradient × local gradient*, summed over outgoing edges. A node used twice receives the sum of the two contributions.

For the unit of Fig. 5 with squared loss $\ell = \frac{1}{2}(a-y)^2$: $\bar{a} = a - y$, $\bar{z} = (a-y)a(1-a)$, $\bar{w} = \bar{z}\,x$. The update of w is proportional to the input x: large inputs produce large weight updates, one reason to standardize features.

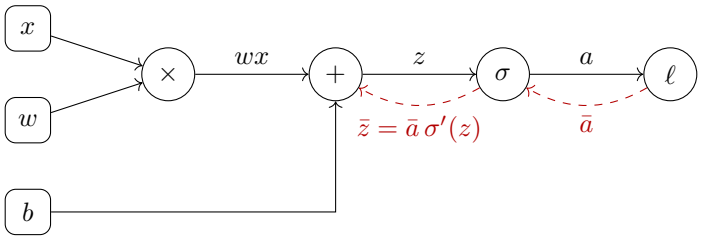


FIGURE 5. Forward values (black) and backward adjoints (red) for a single sigmoid unit. Then $\bar{w} = \bar{z}\,x$, $\bar{b} = \bar{z}$ and $\bar{x} = \bar{z}\,w$.

Theorem 9.2 (Cost of the gradient [14])**.** *If each elementary operation has partial derivatives computable at a cost bounded by a constant multiple of its own cost, then the full gradient of a scalar function computed by a graph can be obtained at a cost bounded by a constant multiple (independent of the number of inputs) of the cost of evaluating the function.*

Sketch. The backward sweep of Proposition 9.1 visits each edge once and performs a multiplication and an addition per edge, plus the local partial derivatives at each node. □

By contrast, *forward-mode* differentiation propagates directional derivatives from inputs to outputs and needs one sweep per input direction; finite differences need one evaluation per parameter. For a network with P parameters and one scalar loss, reverse mode is $\Theta(P)$ times cheaper. The price is memory: the forward values must be stored for the backward pass.

9.2. **Backpropagation in a feedforward network.** Take the network of Section 8 with hidden activation ϕ, softmax output and cross-entropy loss $\ell = -\log \hat{y}_y$, where $\hat{\mathbf{y}} = \operatorname{softmax}(\mathbf{z}^{(L)})$ and $\mathbf{y} = e_y$ is one-hot. Define the *error signals* $\boldsymbol{\delta}^{(l)} = \partial \ell / \partial \mathbf{z}^{(l)} \in \mathbb{R}^{n_l}$.

Theorem 9.3 (Backpropagation)**.**

$$\begin{aligned} \boldsymbol{\delta}^{(L)} &= \hat{\mathbf{y}} - \mathbf{y}, \\ \boldsymbol{\delta}^{(l)} &= \big(W^{(l+1)\top} \boldsymbol{\delta}^{(l+1)}\big) \odot \phi'(\mathbf{z}^{(l)}), \qquad l = L-1, \dots, 1, \\ \nabla_{W^{(l)}} \ell &= \boldsymbol{\delta}^{(l)} \big(\mathbf{a}^{(l-1)}\big)^{\top}, \qquad \nabla_{\mathbf{b}^{(l)}} \ell = \boldsymbol{\delta}^{(l)}. \end{aligned}$$

Proof. The first line is Proposition 8.6. For the recurrence, $\mathbf{z}^{(l+1)} = W^{(l+1)} \phi(\mathbf{z}^{(l)}) + \mathbf{b}^{(l+1)}$, so $\partial z_k^{(l+1)} / \partial z_j^{(l)} = W_{kj}^{(l+1)} \phi'(z_j^{(l)})$, and the chain rule gives $\delta_j^{(l)} = \sum_k \delta_k^{(l+1)} W_{kj}^{(l+1)} \phi'(z_j^{(l)})$. Finally $z_i^{(l)} = \sum_j W_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}$ depends on $W_{ij}^{(l)}$ only through itself, with derivative $a_j^{(l-1)}$, so $\partial \ell / \partial W_{ij}^{(l)} = \delta_i^{(l)} a_j^{(l-1)}$; similarly for the bias. □

For a mini-batch the gradients are averaged: stacking the $\boldsymbol{\delta}^{(l)}$ and $\mathbf{a}^{(l-1)}$ of the B examples as columns of $\Delta^{(l)}$ and $A^{(l-1)}$, $\nabla_{W^{(l)}} = \frac{1}{B} \Delta^{(l)} A^{(l-1)\top}$. For ReLU, $\phi'(z) = \mathbf{1}_{z>0}$ and the recurrence simply zeroes the signal at inactive units; a dead unit never receives gradient and never recovers.

9.3. **Vanishing and exploding gradients.** Iterating the recurrence,

$$\boldsymbol{\delta}^{(l)} = D^{(l)} W^{(l+1)\top} D^{(l+1)} \cdots W^{(L)\top} \boldsymbol{\delta}^{(L)}, \qquad D^{(k)} = \operatorname{diag}\big(\phi'(\mathbf{z}^{(k)})\big),$$

hence

$$\|\boldsymbol{\delta}^{(l)}\| \le \Big(\prod_{k=l}^{L-1} \|D^{(k)}\| \, \|W^{(k+1)}\| \Big) \|\boldsymbol{\delta}^{(L)}\|.$$

For the sigmoid $\|D^{(k)}\| \le \frac{1}{4}$, so unless the weights are large the signal shrinks geometrically with depth: the gradients of early layers *vanish*. With large weights the product can instead *explode*. This explains the preference for ReLU-type activations and the importance of initialization.

Proposition 9.4 (Variance-preserving initialization)**.** *Let a layer compute $z_i = \sum_{j=1}^{n} W_{ij} a_j$ with W_{ij} i.i.d., mean 0, variance v, independent of the inputs a_j, which are i.i.d. Then $\operatorname{Var} z_i = nv\,\mathbb{E}[a_j^2]$. If $a_j = \mathrm{ReLU}(z'_j)$ with z'_j symmetric about 0, then $\mathbb{E}a_j^2 = \frac{1}{2}\operatorname{Var} z'_j$, so $\operatorname{Var} z_i = \operatorname{Var} z'_j$ when $v = 2/n$ (He initialization [15]). For a symmetric activation with $\phi'(0) = 1$ near the linear regime, the analogous choice is $v = 1/n$ (and $v = 2/(n_{\text{in}} + n_{\text{out}})$ balances forward and backward passes: Glorot initialization).*

Proof. $\operatorname{Var} z_i = \sum_j \mathbb{E}[W_{ij}^2]\mathbb{E}[a_j^2] = nv\mathbb{E}a_j^2$ by independence and $\mathbb{E}W_{ij} = 0$. For symmetric z', $\mathbb{E}\,\mathrm{ReLU}(z')^2 = \mathbb{E}[z'^2 \mathbf{1}_{z'>0}] = \frac{1}{2}\mathbb{E}z'^2$. □

9.4. **Gradient checking.** For $f \in C^3$, Taylor's theorem gives $\frac{f(\theta+he_i)-f(\theta-he_i)}{2h} = \partial_i f(\theta) + O(h^2)$, against $O(h)$ for the one-sided quotient. Comparing a backpropagated gradient with central differences, via the relative error $\|g-\tilde{g}\|/(\|g\|+\|\tilde{g}\|)$, detects implementation errors; in double precision $h \approx 10^{-5}$ balances truncation and rounding error.

Exercise 9.5. Generalize Theorem 9.3 to an arbitrary differentiable loss and output activation, and show that the formula $\boldsymbol{\delta}^{(L)} = \hat{\mathbf{y}} - \mathbf{y}$ also holds for a sigmoid output with binary cross-entropy and for an identity output with squared loss $\frac{1}{2}\|\hat{\mathbf{y}} - \mathbf{y}\|^2$. (These are the *canonical link* pairings of generalized linear models.)

Exercise 9.6. For a deep sigmoid network with all weights $W^{(k)} = sQ_k$, Q_k orthogonal, estimate $\|\boldsymbol{\delta}^{(1)}\|/\|\boldsymbol{\delta}^{(L)}\|$ when all pre-activations are near 0, and find the scale s that keeps it of order 1.

10. Architectures of deep networks

Architectures differ in the graph of their computation and, through it, in their inductive bias. We treat the five families of the course: multilayer perceptrons (Section 8), convolutional, recurrent and attention-based networks, and autoencoders.

10.1. **Convolutional networks.** For signals $x\colon \mathbb{Z} \to \mathbb{R}$ and a kernel k supported on $\{0,\dots,K-1\}$, the (cross-)correlation used in deep learning is $(k \star x)_t = \sum_{s=0}^{K-1} k_s x_{t+s}$. A convolutional layer with C_{in} input and C_{out} output channels computes

$$z_{c,t} = b_c + \sum_{c'=1}^{C_{\text{in}}} (k_{c,c'} \star x_{c'})_t,$$

followed by an activation, and often by *pooling* (a maximum or average over windows) or a *stride* (keeping every s-th output). Images use two-dimensional kernels. Let $(T_u x)_t = x_{t-u}$ be translation.

Proposition 10.1. *The following hold.*

- *(a) Convolutional layers are translation equivariant: $k \star (T_u x) = T_u(k \star x)$, and entrywise activations commute with T_u.*
- *(b) A layer has $KC_{\text{in}}C_{\text{out}} + C_{\text{out}}$ parameters, independent of the length of the signal, whereas a dense layer between the same spaces of length N has $N^2 C_{\text{in}} C_{\text{out}} + NC_{\text{out}}$.*
- *(c) The output of L stacked stride-1 layers with kernel size K at position t depends exactly on the inputs at positions $t,\dots,t+L(K-1)$: the* receptive field *has size $1 + L(K-1)$.*

Proof. (a) $(k \star T_u x)_t = \sum_s k_s x_{t+s-u} = (k \star x)_{t-u}$. (b) Counting. (c) By induction: if layer $L-1$ at positions $t, \dots, t+K-1$ depends on inputs $t, \dots, t+K-1+(L-1)(K-1)$, then layer L at t depends on inputs up to $t + L(K-1)$; generic weights make the dependence exact. □

Weight sharing and locality are the inductive bias: the same feature detector is applied everywhere, and features are built hierarchically from local ones. Translation equivariance is a special case of the group equivariance of Section 15.

10.2. **Recurrent networks.** A recurrent network processes a sequence $\mathbf{x}_1, \mathbf{x}_2, \dots$ with a fixed cell

$$\mathbf{h}_t = \phi(W\mathbf{h}_{t-1} + U\mathbf{x}_t + \mathbf{b}), \qquad \hat{\mathbf{y}}_t = g(V\mathbf{h}_t + \mathbf{c}),$$

sharing $(W, U, V, \mathbf{b}, \mathbf{c})$ across time; the *hidden state* $\mathbf{h}_t$ summarizes the past. Since the same finite set of parameters is applied at every step, a recurrent network defines a map on sequences of every length, which a feedforward network on a fixed window cannot do. Training unrolls the recursion into a feedforward graph of depth T and applies Proposition 9.1 (*backpropagation through time*). Then

$$\frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_t} = \prod_{s=t+1}^{T} \operatorname{diag}(\phi'(W\mathbf{h}_{s-1} + U\mathbf{x}_s + \mathbf{b}))W, \qquad \Big\| \frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_t} \Big\| \le (\gamma \|W\|)^{T-t},$$

with $\gamma = \sup|\phi'|$. If $\gamma\|W\| < 1$ the influence of $\mathbf{x}_t$ on the loss at time T decays exponentially in $T-t$: plain recurrent networks cannot learn long-range dependencies. The gated cells of Section 11 were designed to fix this.

10.3. **Attention and Transformers.** Let $X \in \mathbb{R}^{T \times d}$ hold a sequence of T token embeddings as rows. *Self-attention* [17] forms queries $Q = XW_Q$, keys $K = XW_K$ and values $V = XW_V$, with $W_Q, W_K \in \mathbb{R}^{d \times d_k}$, $W_V \in \mathbb{R}^{d \times d_v}$, and outputs

$$\operatorname{Attn}(X) = \operatorname{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}}\Big)V,$$

the softmax acting on each row. Row t of the output is a convex combination of the value vectors, weighted by the similarity of query t with each key: a differentiable, content-addressed lookup. *Multi-head* attention runs H such maps in parallel with $d_k = d/H$ and concatenates them. A *Transformer block* is $X \mapsto Y = X + \mathrm{MultiHead}(\mathrm{LN}(X))$ followed by $Y \mapsto Y + \mathrm{MLP}(\mathrm{LN}(Y))$, the MLP acting on each row, with layer normalization LN. In autoregressive models a *causal mask* sets the logits $QK^\top$ to $-\infty$ above the diagonal, so that position t attends only to positions $\le t$.

Proposition 10.2. *The following hold.*

- *(a) If the entries of $q, k \in \mathbb{R}^{d_k}$ are independent with mean 0 and variance 1, then $\operatorname{Var}\langle q, k\rangle = d_k$; dividing by $\sqrt{d_k}$ keeps the logits of order 1, away from the saturated regime of the softmax.*
- *(b) Self-attention is permutation equivariant: for every permutation matrix P, $\operatorname{Attn}(PX) = P\operatorname{Attn}(X)$. So is a Transformer block.*
- *(c) The cost of self-attention is $O(T^2 d)$ time and $O(T^2)$ memory for the attention matrix.*

Proof. (a) $\operatorname{Var}\sum_i q_i k_i = \sum_i \mathbb{E}q_i^2 \mathbb{E}k_i^2 = d_k$. (b) Q, K, V become PQ, PK, PV, the logits become $PQK^\top P^\top$, the row-wise softmax commutes with simultaneous permutation of rows and columns, and $P\operatorname{softmax}(\cdot)P^\top PV = P\operatorname{softmax}(\cdot)V$. Row-wise maps and residual connections are also equivariant. (c) Forming $QK^\top$ dominates. □

By (b), a Transformer without positional information treats its input as a multiset: it cannot tell "dog bites man" from "man bites dog". Order is supplied by adding *positional encodings* to X, for instance $p_{t,2i} = \sin(t/10000^{2i/d})$, $p_{t,2i+1} = \cos(t/10000^{2i/d})$, for which p_{t+u} is a fixed linear (rotation) function of p_t for each offset u; or learned position vectors; or rotations applied directly to queries and keys.

10.4. **Autoencoders.** An *autoencoder* is a pair of maps, an encoder $E\colon \mathbb{R}^n \to \mathbb{R}^k$ and a decoder $D\colon \mathbb{R}^k \to \mathbb{R}^n$ with $k < n$, trained to minimize the reconstruction error $\sum_i \|x_i - D(E(x_i))\|^2$. The bottleneck forces E to keep the most informative features.

Theorem 10.3 (Linear autoencoders are PCA [18])**.** *Let E, D be linear and the data centred, with data matrix $X \in \mathbb{R}^{N\times n}$ and SVD $X = U\Sigma V^\top$. The minimum of $\|X - XE^\top D^\top\|_F^2$ over $E \in \mathbb{R}^{k\times n}$, $D \in \mathbb{R}^{n\times k}$ equals $\sum_{i>k}\sigma_i^2$, attained when DE is the orthogonal projection onto* $\operatorname{span}(v_1, \dots, v_k)$*. If $\sigma_k > \sigma_{k+1}$, every minimizer has* $\operatorname{im} D = \operatorname{span}(v_1, \dots, v_k)$*.*

Proof. $XE^\top D^\top$ has rank at most k, so Theorem 2.4 gives the lower bound; the projection attains it by Proposition 2.5. If $\sigma_k > \sigma_{k+1}$, the Eckart–Young minimizer X_k is unique, so $XE^\top D^\top = X_k$, whose row space is $\operatorname{span}(v_1, \dots, v_k)$ and is contained in $\operatorname{im} D$; dimensions force equality. □

Nonlinear autoencoders learn curved low-dimensional representations, but their latent spaces have no prescribed distribution and cannot be sampled from; variational autoencoders (Section 12) remedy this.

Exercise 10.4. Compare a dense layer and a convolutional layer on signals of length $N = 100$ with $C_{\text{in}} = C_{\text{out}} = 16$ and $K = 5$: count parameters and multiply–add operations, and describe which functions the dense layer can represent that the convolutional one cannot.

Exercise 10.5. Design a classification task on sequences of length T that a Transformer without positional encodings cannot solve (for example: "is the first element equal to the last?") and prove that it cannot, using Proposition 10.2(b) and the fact that the classifier pools the output rows symmetrically.

Exercise 10.6. Prove that the sinusoidal encodings satisfy $\left(\begin{smallmatrix} p_{t+u,2i} \\ p_{t+u,2i+1}\end{smallmatrix}\right) = \left(\begin{smallmatrix} \cos u\omega_i & \sin u\omega_i \\ -\sin u\omega_i & \cos u\omega_i\end{smallmatrix}\right)\left(\begin{smallmatrix} p_{t,2i} \\ p_{t,2i+1}\end{smallmatrix}\right)$ with $\omega_i = 10000^{-2i/d}$, a rotation independent of t.

11. Memory in sequence models

A sequence model must decide what to retain of the past. Three answers dominate: gated recurrent cells, which keep a fixed-size state and learn what to write and erase; attention, which keeps everything and looks it up; and state-space models, which keep a fixed-size state evolving linearly, possibly with input-dependent dynamics.

11.1. **Gated recurrent cells.** The LSTM [16] keeps a *cell state* $\mathbf{c}_t$ next to the hidden state $\mathbf{h}_t$. With σ the sigmoid and $[\cdot,\cdot]$ concatenation,

$$\mathbf{f}_t = \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f), \quad \mathbf{i}_t = \sigma(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i), \quad \mathbf{o}_t = \sigma(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o),$$
$$\tilde{\mathbf{c}}_t = \tanh(W_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c), \qquad \mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh \mathbf{c}_t.$$

The forget gate $\mathbf{f}_t$ decides what to keep, the input gate $\mathbf{i}_t$ what to write, the output gate $\mathbf{o}_t$ what to expose.

Proposition 11.1 (Constant error carousel)**.** *Along the direct path through the cell states, holding the gates fixed,*

$$\frac{\partial \mathbf{c}_T}{\partial \mathbf{c}_t} = \prod_{s=t+1}^{T} \operatorname{diag}(\mathbf{f}_s).$$

Proof. $\partial \mathbf{c}_s / \partial \mathbf{c}_{s-1} = \operatorname{diag}(\mathbf{f}_s)$ when $\mathbf{f}_s, \mathbf{i}_s, \tilde{\mathbf{c}}_s$ are treated as constants; multiply. □

When a forget gate is close to 1 the corresponding factor is close to 1 and the gradient travels across long spans without decay. In a plain recurrent network the factors are $\operatorname{diag}(\phi'(\cdot))W$, whose product decays or explodes geometrically. The decay in the LSTM is learned and input dependent; the full Jacobian contains further terms through the dependence of the gates on $\mathbf{h}_{s-1}$, which do not cancel this path. The GRU merges the forget and input gates into one, $\mathbf{h}_t = (1 - \mathbf{u}_t) \odot \mathbf{h}_{t-1} + \mathbf{u}_t \odot \tilde{\mathbf{h}}_t$, with fewer parameters and the same mechanism.

11.2. **Attention as memory: the key–value cache.** In a causal Transformer the output at position t is $\sum_{s \le t} \alpha_{ts} \mathbf{v}_s$ with $\alpha_{ts} \propto \exp(\langle \mathbf{q}_t, \mathbf{k}_s \rangle / \sqrt{d_k})$: a soft dictionary lookup in which the past is stored verbatim as key–value pairs. During generation the keys and values of past tokens do not change, so they are cached and each new token costs $O(td)$ instead of recomputing $O(t^2 d)$. The memory grows linearly with the context.

Proposition 11.2. *A Transformer with L layers, H heads of key and value dimension d_k, stored with b bytes per number, needs $2LHd_kbT$ bytes of cache for a context of T tokens per sequence.*

For $L = H = 32$, $d_k = 128$ and 16-bit numbers ($b = 2$), this is 524,288 bytes per token: 4 GiB at $T = 8{,}192$, and the cache reaches 16 GiB at $T = 32{,}768$. Managing this cache (sharing keys across heads, quantizing, evicting tokens) is a large part of the engineering of language models [19].

11.3. **State-space models.** A linear *state-space model* (SSM) is

$$\mathbf{h}_t = \bar{A}\mathbf{h}_{t-1} + \bar{B}x_t, \qquad y_t = C\mathbf{h}_t,$$

with $\mathbf{h}_t \in \mathbb{R}^N$. It is usually obtained from a continuous-time system $\dot{\mathbf{h}} = A\mathbf{h} + Bx$ by the zero-order-hold discretization with step Δ: $\bar{A} = e^{\Delta A}$, $\bar{B} = A^{-1}(e^{\Delta A} - I)B$.

Proposition 11.3 (Recurrence–convolution duality)**.** *With $\mathbf{h}_0 = 0$, $y_t = \sum_{s=0}^{t-1} K_s x_{t-s}$ where $K_s = C\bar{A}^s\bar{B}$. For a scalar state with $|a| < 1$, $K_s = c\,a^s\,b$ decays exponentially, with half-life $\log 2 / \log(1/|a|)$ steps.*

Proof. Unrolling, $\mathbf{h}_t = \sum_{s=0}^{t-1} \bar{A}^s \bar{B} x_{t-s}$; apply C. □

The half-life is about 69 steps for $a = 0.99$ and 6.6 steps for $a = 0.9$: the spectrum of $\bar{A}$ sets the memory horizon. The duality gives the best of both worlds: training computes the convolution in parallel over the whole sequence (in $O(T \log T)$ by the fast Fourier transform), while generation runs the recurrence with constant memory per step.

11.4. **Selective state spaces.** A time-invariant SSM applies the same dynamics to every token and so cannot decide, based on content, to ignore a token or to reset its memory. *Mamba* [21] makes Δ, B and C functions of the current input: $\mathbf{h}_t = \bar{A}_t\mathbf{h}_{t-1} + \bar{B}_t x_t$, $y_t = C_t\mathbf{h}_t$. A large Δ_t with A stable drives $\bar{A}_t = e^{\Delta_t A}$ towards 0 (reset and focus on the current token); a small Δ_t gives $\bar{A}_t \approx I$ (ignore the token and keep the state). The model is no longer a convolution, but remains linear in $\mathbf{h}$ and can be computed in parallel.

Proposition 11.4 (Parallel scan)**.** *The maps $\psi_t(\mathbf{h}) = \bar{A}_t\mathbf{h} + \bar{B}_t x_t$ compose as affine maps, $(A_2, b_2) \circ (A_1, b_1) = (A_2A_1, A_2b_1 + b_2)$, and composition is associative. Hence all states $\mathbf{h}_t = \psi_t \circ \cdots \circ \psi_1(0)$ can be computed by a prefix scan in $O(\log T)$ parallel steps and $O(T)$ total work.*

Selective SSMs thus have linear cost in T and constant memory at inference, against the quadratic cost and linear memory of attention, at the price of compressing the past into a fixed-size state; which tasks require exact recall of the past, and therefore attention, is the subject of current research [20].

Exercise 11.5. Prove Proposition 11.2 and compute the cache size when the 32 query heads share 8 key–value heads (grouped-query attention).

Exercise 11.6. For the diagonal SSM $\bar{A} = \operatorname{diag}(a_1, \dots, a_N)$ with $0 < a_i < 1$, show that K_s is a sum of N exponentials, and show that any finite sequence $K_0, \dots, K_M$ can be matched exactly by some diagonal SSM with $N = M + 1$ (complex a_i allowed).

12. Generative models

12.1. **The problem.** A *discriminative* model learns $p(y \mid x)$; a *generative* model learns $p(x)$ (or $p(x, y)$) and can sample from it. Three strategies dominate: *latent variable models*, which push a simple distribution through a learned map; *autoregressive models*, which factor $p(x)$ by the chain rule; and *diffusion models* (Section 13), which learn to reverse a noising process.

12.2. **Probabilistic PCA.** Let $\mathbf{z} \sim \mathcal{N}(0, I_k)$ and $\mathbf{x} = W\mathbf{z} + \mu + \boldsymbol{\varepsilon}$ with $\boldsymbol{\varepsilon} \sim \mathcal{N}(0, \sigma^2 I_n)$ independent. Then $\mathbf{x} \sim \mathcal{N}(\mu, WW^\top + \sigma^2 I)$, and $\mathbf{z} \mid \mathbf{x} \sim \mathcal{N}(M^{-1}W^\top(\mathbf{x} - \mu), \sigma^2 M^{-1})$ with $M = W^\top W + \sigma^2 I$ (Proposition 3.7). Tipping and Bishop [22] showed that the maximum likelihood estimate is $\hat{W} = V_k(\Lambda_k - \hat{\sigma}^2 I)^{1/2} R$, where V_k, Λ_k hold the top k eigenvectors and eigenvalues of the sample covariance, R is any orthogonal matrix and $\hat{\sigma}^2$ is the average of the discarded eigenvalues. PCA is thus the zero-noise limit of a generative model; sampling it produces new points near the principal subspace.

12.3. **Latent variable models and the ELBO.** Let $p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} \mid \mathbf{z}) p(\mathbf{z})\, d\mathbf{z}$ with prior $p(\mathbf{z}) = \mathcal{N}(0, I)$ and a decoder $p_\theta(\mathbf{x} \mid \mathbf{z})$ given by a neural network. The integral is intractable, and so is the posterior $p_\theta(\mathbf{z} \mid \mathbf{x})$. Variational inference introduces an *encoder* $q_\phi(\mathbf{z} \mid \mathbf{x})$, another network, typically $\mathcal{N}(\mu_\phi(\mathbf{x}), \operatorname{diag} \sigma_\phi^2(\mathbf{x}))$.

Theorem 12.1 (Evidence lower bound)**.** *For every* $\mathbf{x}$*,*

$$\log p_\theta(\mathbf{x}) = \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}\mid\mathbf{x})}\big[\log p_\theta(\mathbf{x} \mid \mathbf{z})\big] - D_{\mathrm{KL}}\big(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z})\big)}_{\mathrm{ELBO}(\theta,\phi;\mathbf{x})} + D_{\mathrm{KL}}\big(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p_\theta(\mathbf{z} \mid \mathbf{x})\big).$$

In particular $\log p_\theta(\mathbf{x}) \geq \mathrm{ELBO}$*, with equality if and only if* $q_\phi(\cdot \mid \mathbf{x}) = p_\theta(\cdot \mid \mathbf{x})$*.*

Proof. The inequality alone follows from Jensen:

$$\log p_\theta(\mathbf{x}) = \log \mathbb{E}_q\Big[\frac{p_\theta(\mathbf{x} \mid \mathbf{z}) p(\mathbf{z})}{q(\mathbf{z} \mid \mathbf{x})}\Big] \geq \mathbb{E}_q \log \frac{p_\theta(\mathbf{x} \mid \mathbf{z}) p(\mathbf{z})}{q(\mathbf{z} \mid \mathbf{x})} = \mathrm{ELBO}.$$

For the identity, write $p_\theta(\mathbf{x} \mid \mathbf{z})p(\mathbf{z}) = p_\theta(\mathbf{z} \mid \mathbf{x})p_\theta(\mathbf{x})$, so $\mathrm{ELBO} = \mathbb{E}_q \log \frac{p_\theta(\mathbf{z}\mid\mathbf{x}) p_\theta(\mathbf{x})}{q(\mathbf{z}\mid\mathbf{x})} = \log p_\theta(\mathbf{x}) - D_{\mathrm{KL}}(q \| p_\theta(\cdot \mid \mathbf{x}))$. Equality follows from Theorem 4.2. □

A *variational autoencoder* (VAE) [23] maximizes $\sum_i \mathrm{ELBO}(\theta, \phi; \mathbf{x}_i)$ jointly in θ and ϕ. The first term rewards reconstruction; the second keeps the encoded distributions close to the prior, so that decoding samples $\mathbf{z} \sim \mathcal{N}(0, I)$ produces realistic data. By Proposition 4.4, for a diagonal Gaussian encoder,

$$D_{\mathrm{KL}}\big(\mathcal{N}(\mu, \operatorname{diag}\sigma^2) \,\|\, \mathcal{N}(0, I)\big) = \frac{1}{2}\sum_{j=1}^{k} (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2).$$

The reconstruction term fixes the loss: a Bernoulli decoder (pixels in $[0, 1]$) gives the binary cross-entropy, a Gaussian decoder $\mathcal{N}(\hat{\mathbf{x}}, s^2 I)$ gives $\frac{1}{2s^2}\|\mathbf{x} - \hat{\mathbf{x}}\|^2$. The Gaussian decoder's mean is the conditional average of all images compatible with $\mathbf{z}$, which explains the blurriness of mean-squared-error reconstructions. Multiplying the divergence by a weight β trades reconstruction (β small: sharp reconstructions, a latent space with holes, poor samples) against regularity (β large:

a latent space filled like the prior, blurrier outputs, and eventually *posterior collapse*, where $q_\phi(\mathbf{z} \mid \mathbf{x}) = p(\mathbf{z})$ and $\mathbf{z}$ carries no information).

12.4. **The reparameterization trick.** The gradient in ϕ of $\mathbb{E}_{q_\phi}[f(\mathbf{z})]$ cannot be taken inside the expectation, since the distribution depends on ϕ. Writing $\mathbf{z} = \mu_\phi(\mathbf{x}) + \sigma_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}$ with $\boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$ moves the dependence into the integrand:

$$\nabla_\phi \mathbb{E}_{q_\phi}[f(\mathbf{z})] = \mathbb{E}_{\boldsymbol{\epsilon}}\big[\nabla_\phi f(\mu_\phi + \sigma_\phi \odot \boldsymbol{\epsilon})\big],$$

valid when f is differentiable with integrable derivative (dominated convergence). A single sample of $\boldsymbol{\epsilon}$ gives an unbiased low-variance estimate, and backpropagation goes through the sampling step.

12.5. **Topology of the latent space.** Data concentrated near a d-dimensional manifold $\mathcal{M}$ need a latent space into which $\mathcal{M}$ embeds, if the encoder and decoder are continuous and reconstruct exactly.

Proposition 12.2. *If $E\colon \mathcal{M} \to \mathbb{R}^k$ and $D\colon \mathbb{R}^k \to \mathbb{R}^n$ are continuous with $D \circ E = \mathrm{id}_{\mathcal{M}}$ and $\mathcal{M}$ is compact, then E is an embedding. Consequently:*

(a) for the circle S^1, $k = 1$ is impossible and $k = 2$ suffices;

(b) for the torus $T^2 = S^1 \times S^1$, $k = 2$ is impossible and $k = 3$ suffices.

Proof. $D \circ E = \mathrm{id}$ forces E injective, and a continuous injection from a compact space to a Hausdorff space is a homeomorphism onto its image. (a) If $E\colon S^1 \to \mathbb{R}$ were injective, $E(S^1)$ would be a compact interval $[a, b]$ with $a < b$; pick p with $E(p) \in (a, b)$. Then $S^1 \setminus \{p\}$ is connected but its image $[a, b] \setminus \{E(p)\}$ is not. (b) If T^2 embedded in $\mathbb{R}^2$, by invariance of domain the image of the compact surface would be open in $\mathbb{R}^2$; being also compact, hence closed, and nonempty, it would be all of $\mathbb{R}^2$, which is not compact. The standard torus of revolution embeds in $\mathbb{R}^3$. □

A VAE with a one-dimensional latent space trained on points of a circle must therefore "cut" the circle somewhere, and reconstructions near the cut are poor; periodic latent variables (an angle) avoid the cut.

12.6. **Normalizing flows.** If $f_\theta\colon \mathbb{R}^n \to \mathbb{R}^n$ is a diffeomorphism and $\mathbf{x} = f_\theta(\mathbf{z})$ with $\mathbf{z} \sim \mathcal{N}(0, I)$, then by Theorem 3.3

$$\log p_\theta(\mathbf{x}) = \log \mathcal{N}\big(f_\theta^{-1}(\mathbf{x}); 0, I\big) + \log\big|\det D f_\theta^{-1}(\mathbf{x})\big|,$$

which is exact and can be maximized directly [24]. Flows compose simple invertible layers whose Jacobian determinants are cheap, for example *coupling layers* $(\mathbf{x}_1, \mathbf{x}_2) \mapsto (\mathbf{x}_1, \mathbf{x}_2 \odot e^{s(\mathbf{x}_1)} + t(\mathbf{x}_1))$, whose Jacobian is triangular with determinant $e^{\sum s(\mathbf{x}_1)}$.

12.7. **Autoregressive models.** By the chain rule of probability, every distribution on sequences factors as

$$p(x_1, \dots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_1, \dots, x_{t-1}).$$

An autoregressive model parametrizes each factor by a network, in large language models a causal Transformer (Section 10) emitting a softmax over a vocabulary. Training maximizes the exact log-likelihood, i.e. minimizes the next-token cross-entropy on real prefixes (*teacher forcing*); quality is reported as the *perplexity* exp(cross-entropy per token). Generation samples one token at a time, so a length-T output needs T sequential forward passes (each cheap with the cache of Proposition 11.2), against one pass for a VAE.

Sampling uses the temperature softmax $p_\tau(i) = e^{z_i/\tau} / \sum_j e^{z_j/\tau}$.

Proposition 12.3. *The entropy $H(p_\tau)$ is nondecreasing in $\tau \in (0,\infty)$, strictly increasing unless all logits are equal; it tends to $\log K$ as $\tau \to \infty$ and to $\log(\#\arg\max z)$ as $\tau \to 0^+$.*

Proof. Let $\beta = 1/\tau$ and $Z(\beta) = \sum_j e^{\beta z_j}$. Then $\log p_\tau(i) = \beta z_i - \log Z$, so $H = \log Z(\beta) - \beta \mathbb{E}_\beta[z]$, where $\mathbb{E}_\beta$ is the expectation under p_τ. Since $\frac{d}{d\beta}\log Z = \mathbb{E}_\beta[z]$ and $\frac{d}{d\beta}\mathbb{E}_\beta[z] = \operatorname{Var}_\beta(z)$,

$$\frac{dH}{d\beta} = \mathbb{E}_\beta[z] - \mathbb{E}_\beta[z] - \beta \operatorname{Var}_\beta(z) = -\beta \operatorname{Var}_\beta(z) \le 0,$$

with equality only if z is constant. As β decreases (i.e. τ increases) the entropy increases. The limits follow from the limits of p_τ noted in Section 8. □

Exercise 12.4. Derive the posterior $p(\mathbf{z} \mid \mathbf{x})$ of probabilistic PCA and show that, as $\sigma \to 0$, its mean tends to the orthogonal projection coordinates $(W^\top W)^{-1} W^\top (\mathbf{x} - \mu)$.

Exercise 12.5. Sketch how a trained VAE could classify digits through Bayes' rule, using one VAE per class and the ELBO as a surrogate for $\log p(\mathbf{x} \mid y)$. Why should one not expect it to beat a discriminative classifier with the same number of parameters?

Exercise 12.6. Show that the ELBO gap $D_{\mathrm{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \| p_\theta(\mathbf{z} \mid \mathbf{x}))$ can be made zero for probabilistic PCA with a Gaussian encoder, and compute the optimal encoder.

13. Diffusion models and score-based generation

13.1. **The forward process.** Fix a *noise schedule* $0 < \beta_1 < \cdots < \beta_T < 1$ and put $\alpha_t = 1 - \beta_t$, $\bar\alpha_t = \prod_{s \le t} \alpha_s$. Starting from a data point $\mathbf{x}_0 \sim p_{\mathrm{data}}$, the forward process is the Markov chain

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\big(\sqrt{\alpha_t}\,\mathbf{x}_{t-1},\ \beta_t I\big).$$

Proposition 13.1. $q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\big(\sqrt{\bar\alpha_t}\,\mathbf{x}_0, (1-\bar\alpha_t) I\big)$; *equivalently* $\mathbf{x}_t = \sqrt{\bar\alpha_t}\mathbf{x}_0 + \sqrt{1-\bar\alpha_t}\,\boldsymbol{\epsilon}$ *with* $\boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$.

Proof. By induction: if $\mathbf{x}_{t-1} = \sqrt{\bar\alpha_{t-1}}\mathbf{x}_0 + \sqrt{1-\bar\alpha_{t-1}}\boldsymbol{\epsilon}'$, then $\mathbf{x}_t = \sqrt{\alpha_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon}''$ with independent standard normals, and the noise is normal with variance $\alpha_t - \bar\alpha_t + 1 - \alpha_t = 1 - \bar\alpha_t$. □

If $\bar\alpha_T \approx 0$, $\mathbf{x}_T$ is nearly $\mathcal{N}(0, I)$ whatever the data: the forward process destroys all structure. Generation runs it backwards.

13.2. **Scores and Langevin dynamics.** The *score* of a density p is $\nabla_{\mathbf{x}} \log p(\mathbf{x})$; it does not depend on the normalizing constant. For $\mathcal{N}(\mu, \sigma^2 I)$ it is $-(\mathbf{x} - \mu)/\sigma^2$.

Proposition 13.2 (Langevin dynamics)**.** *Let $p > 0$ be a smooth density with $\nabla \log p$ Lipschitz. The diffusion $d\mathbf{x}_t = \tfrac{1}{2}\nabla \log p(\mathbf{x}_t)\,dt + d\mathbf{W}_t$ has p as a stationary density.*

Proof. The Fokker–Planck equation of the diffusion is $\partial_t \rho = -\nabla \cdot (\tfrac{1}{2}\rho \nabla \log p) + \tfrac{1}{2}\Delta\rho$. For $\rho = p$ the right side is $-\tfrac{1}{2}\nabla \cdot \nabla p + \tfrac{1}{2}\Delta p = 0$. □

Its discretization $\mathbf{x}_{k+1} = \mathbf{x}_k + \tfrac{\eta}{2}\nabla \log p(\mathbf{x}_k) + \sqrt{\eta}\,\boldsymbol{\xi}_k$ samples approximately from p using only the score. Learning the score is therefore enough to generate. The difficulty is that a score learned from data is inaccurate in low-density regions, which the chain must cross; noising the data at many scales fills those regions.

13.3. **Tweedie's formula and denoising score matching.** Let q_t be the marginal density of $\mathbf{x}_t$.

Theorem 13.3 (Tweedie)**.**

$$\nabla \log q_t(\mathbf{x}) = \mathbb{E}\big[\nabla_{\mathbf{x}} \log q(\mathbf{x}_t \mid \mathbf{x}_0) \mid \mathbf{x}_t = \mathbf{x}\big] = -\frac{\mathbf{x} - \sqrt{\bar\alpha_t}\,\mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}_t = \mathbf{x}]}{1-\bar\alpha_t}.$$

Equivalently, $\mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}_t = \mathbf{x}] = \big(\mathbf{x} + (1-\bar\alpha_t)\nabla \log q_t(\mathbf{x})\big)/\sqrt{\bar\alpha_t}$*: the optimal denoiser is determined by the score.*

Proof. $q_t(\mathbf{x}) = \int q(\mathbf{x} \mid \mathbf{x}_0) p_{\text{data}}(\mathbf{x}_0)\, d\mathbf{x}_0$; differentiating under the integral, $\nabla q_t(\mathbf{x}) = \int \nabla_{\mathbf{x}} \log q(\mathbf{x} \mid \mathbf{x}_0)\, q(\mathbf{x} \mid \mathbf{x}_0) p_{\text{data}}(\mathbf{x}_0)\, d\mathbf{x}_0$, and dividing by $q_t(\mathbf{x})$ gives the conditional expectation. Insert $\nabla_{\mathbf{x}} \log q(\mathbf{x} \mid \mathbf{x}_0) = -(\mathbf{x} - \sqrt{\bar\alpha_t}\mathbf{x}_0)/(1-\bar\alpha_t)$. □

Theorem 13.4 (Denoising score matching [26])**.** *For any* $s\colon \mathbb{R}^n \to \mathbb{R}^n$ *with the expectations finite,*

$$\mathbb{E}\big\|s(\mathbf{x}_t) - \nabla \log q(\mathbf{x}_t \mid \mathbf{x}_0)\big\|^2 = \mathbb{E}\big\|s(\mathbf{x}_t) - \nabla \log q_t(\mathbf{x}_t)\big\|^2 + C,$$

where C *does not depend on* s*. Hence regressing on the tractable conditional score learns the intractable marginal score.*

Proof. Expand both squares. The terms $\mathbb{E}\|s\|^2$ coincide, the terms not involving s go into C, and the cross terms agree because, conditioning on $\mathbf{x}_t$ and using Theorem 13.3, $\mathbb{E}\langle s(\mathbf{x}_t), \nabla \log q(\mathbf{x}_t \mid \mathbf{x}_0)\rangle = \mathbb{E}\langle s(\mathbf{x}_t), \mathbb{E}[\nabla \log q(\mathbf{x}_t \mid \mathbf{x}_0) \mid \mathbf{x}_t]\rangle = \mathbb{E}\langle s(\mathbf{x}_t), \nabla \log q_t(\mathbf{x}_t)\rangle$. □

Since $\nabla \log q(\mathbf{x}_t \mid \mathbf{x}_0) = -\boldsymbol{\epsilon}/\sqrt{1-\bar\alpha_t}$ in the notation of Proposition 13.1, parametrizing $s_\theta(\mathbf{x}, t) = -\boldsymbol{\epsilon}_\theta(\mathbf{x}, t)/\sqrt{1-\bar\alpha_t}$ turns the objective, up to weights, into the DDPM loss [25]

$$\mathcal{L}(\theta) = \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}}\big\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\big(\sqrt{\bar\alpha_t}\mathbf{x}_0 + \sqrt{1-\bar\alpha_t}\boldsymbol{\epsilon},\, t\big)\big\|^2 :$$

the network learns to predict the noise that was added.

13.4. **The reverse process.**

Proposition 13.5. $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\tilde\mu_t, \tilde\beta_t I)$ *with*

$$\tilde\mu_t = \frac{\sqrt{\bar\alpha_{t-1}}\beta_t}{1-\bar\alpha_t}\mathbf{x}_0 + \frac{\sqrt{\alpha_t}(1-\bar\alpha_{t-1})}{1-\bar\alpha_t}\mathbf{x}_t, \qquad \tilde\beta_t = \frac{1-\bar\alpha_{t-1}}{1-\bar\alpha_t}\beta_t.$$

Proof. Given $\mathbf{x}_0$, the pair $(\mathbf{x}_{t-1}, \mathbf{x}_t)$ is jointly normal with $\mathbf{x}_{t-1} \sim \mathcal{N}(\sqrt{\bar\alpha_{t-1}}\mathbf{x}_0, (1-\bar\alpha_{t-1})I)$ and $\mathbf{x}_t = \sqrt{\alpha_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon}$. Apply Proposition 3.7 coordinatewise with $\Sigma_{11} = 1-\bar\alpha_{t-1}$, $\Sigma_{12} = \sqrt{\alpha_t}(1-\bar\alpha_{t-1})$, $\Sigma_{22} = 1-\bar\alpha_t$. □

Substituting the estimate $\hat{\mathbf{x}}_0 = (\mathbf{x}_t - \sqrt{1-\bar\alpha_t}\,\boldsymbol{\epsilon}_\theta)/\sqrt{\bar\alpha_t}$ into $\tilde\mu_t$ gives the sampler: draw $\mathbf{x}_T \sim \mathcal{N}(0, I)$ and for $t = T, \dots, 1$ set

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\Big(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar\alpha_t}}\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\Big) + \sqrt{\tilde\beta_t}\,\mathbf{z}_t, \qquad \mathbf{z}_t \sim \mathcal{N}(0, I),$$

with $\mathbf{z}_1 = 0$. The whole procedure maximizes a variational lower bound on $\log p_\theta(\mathbf{x}_0)$, of which the loss above is a reweighting; a diffusion model is a hierarchical VAE whose encoder is fixed.

13.5. **Continuous time.** As $T \to \infty$ with $\beta_t = b(t)\,dt$, the forward chain becomes the SDE $d\mathbf{x} = -\frac{1}{2}b(t)\mathbf{x}\,dt + \sqrt{b(t)}\,d\mathbf{W}$. By Anderson's theorem [28] its time reversal is again a diffusion,

$$d\mathbf{x} = \Big[-\tfrac{1}{2}b(t)\mathbf{x} - b(t)\nabla \log q_t(\mathbf{x})\Big]dt + \sqrt{b(t)}\,d\bar{\mathbf{W}},$$

run backwards in time, and the deterministic *probability-flow ODE* $\dot{\mathbf{x}} = -\frac{1}{2}b(t)\big[\mathbf{x} + \nabla \log q_t(\mathbf{x})\big]$ has the same marginals q_t [27]. Both require only the score. The ODE yields deterministic samplers with few steps (DDIM) and exact likelihoods through the instantaneous change of variables $\frac{d}{dt}\log q_t(\mathbf{x}_t) = -\nabla \cdot \dot{\mathbf{x}}$.

Exercise 13.6. For $p_{\text{data}} = \mathcal{N}(m, s^2)$ in one dimension, compute q_t, its score and $\mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}_t]$ explicitly, and verify Theorem 13.3.

Exercise 13.7. Show that the probability-flow ODE and the forward SDE have the same marginals by checking that both satisfy the same continuity (Fokker–Planck) equation.

14. Reinforcement learning

14.1. **Markov decision processes.** A *Markov decision process* (MDP) is a tuple [29] $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$: finite sets of states and actions, transition probabilities $P(s' \mid s, a)$, a bounded reward $r(s, a)$ and a discount $\gamma \in [0, 1)$. A (stationary, stochastic) *policy* $\pi(a \mid s)$ generates trajectories $s_0, a_0, s_1, a_1, \dots$ with $a_t \sim \pi(\cdot \mid s_t)$, $s_{t+1} \sim P(\cdot \mid s_t, a_t)$. Its *value* and *action-value* functions are

$$V^\pi(s) = \mathbb{E}_\pi\Big[\sum_{t \ge 0} \gamma^t r(s_t, a_t) \;\Big|\; s_0 = s\Big], \qquad Q^\pi(s, a) = r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V^\pi(s').$$

Unlike supervised learning, there are no labels: the agent observes only rewards, which may be delayed, and its own actions determine which data it sees.

14.2. **Bellman equations.** Conditioning on the first step gives the *Bellman equation* $V^\pi = r_\pi + \gamma P_\pi V^\pi$, with $r_\pi(s) = \sum_a \pi(a \mid s) r(s, a)$ and $P_\pi(s' \mid s) = \sum_a \pi(a \mid s) P(s' \mid s, a)$; since $\|\gamma P_\pi\|_\infty = \gamma < 1$, it has the unique solution $V^\pi = (I - \gamma P_\pi)^{-1} r_\pi$. The *Bellman optimality operator* on $\mathbb{R}^{\mathcal{S}}$ is

$$(\mathcal{T}V)(s) = \max_a \Big[r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V(s')\Big].$$

Theorem 14.1. *$\mathcal{T}$ is a γ-contraction in the sup norm. Its unique fixed point V^* satisfies $V^* = \max_\pi V^\pi$ pointwise, any policy greedy with respect to V^* is optimal, and* value iteration *$V_{k+1} = \mathcal{T}V_k$ converges to V^* with $\|V_k - V^*\|_\infty \le \gamma^k \|V_0 - V^*\|_\infty$.*

Proof. For each s, $|\max_a f(a) - \max_a g(a)| \le \max_a |f(a) - g(a)|$, so $|(\mathcal{T}V)(s) - (\mathcal{T}W)(s)| \le \gamma \max_a \sum_{s'} P(s' \mid s, a)|V(s') - W(s')| \le \gamma \|V - W\|_\infty$. Banach's fixed point theorem gives existence, uniqueness and the rate. For any π, $V^\pi = \mathcal{T}_\pi V^\pi$ where $\mathcal{T}_\pi \le \mathcal{T}$ is monotone and contracting; iterating $\mathcal{T}_\pi$ from V^* gives a nonincreasing sequence ($\mathcal{T}_\pi V^* \le \mathcal{T}V^* = V^*$) converging to V^π, so $V^\pi \le V^*$. For π^* greedy with respect to V^*, $\mathcal{T}_{\pi^*} V^* = \mathcal{T}V^* = V^*$, so $V^{\pi^*} = V^*$. □

14.3. **Policy gradients.** Value iteration needs the model P and a table over $\mathcal{S}$. For large or continuous problems one parametrizes a policy π_θ directly, for instance by a network with softmax output, and maximizes $J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[R(\tau)]$, where $\tau = (s_0, a_0, \dots, s_H)$ and $R(\tau) = \sum_t \gamma^t r(s_t, a_t)$.

Theorem 14.2 (Policy gradient [30])**.**

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, G_t\Big], \qquad G_t = \sum_{t' \ge t} \gamma^{t'} r(s_{t'}, a_{t'}),$$

and G_t may be replaced by $G_t - b(s_t)$ for any baseline b depending only on the state, without changing the expectation.

Proof. $p_\theta(\tau) = \rho(s_0)\prod_t \pi_\theta(a_t \mid s_t)P(s_{t+1} \mid s_t, a_t)$, so $\nabla_\theta \log p_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$: the unknown dynamics drop out. The log-derivative identity $\nabla_\theta \mathbb{E}_{p_\theta}[R] = \mathbb{E}_{p_\theta}[R\nabla_\theta \log p_\theta]$ gives the formula with $R(\tau)$ in place of G_t. Rewards earned before time t can be dropped from the t-th term because, conditionally on the history up to s_t, $\mathbb{E}[\nabla_\theta \log \pi_\theta(a_t \mid s_t)] = \sum_a \nabla_\theta \pi_\theta(a \mid s_t) = \nabla_\theta 1 = 0$; the same computation shows the baseline term has mean zero. □

Sampling trajectories and following this estimate is the REINFORCE algorithm; the baseline (typically a learned estimate of V^π) reduces its variance, and $G_t - V(s_t)$ estimates the *advantage* $Q^\pi - V^\pi$. Actor–critic methods, PPO, and deep Q-learning, which fits Q_θ to the targets $r + \gamma \max_{a'} Q_{\theta^-}(s', a')$, scale these ideas with neural networks.

14.4. **Learning from human preferences.** Language models are aligned with human preferences in three stages: supervised fine-tuning on demonstrations, learning a reward model from comparisons, and optimizing the policy against it. Given a prompt x and two responses with y_w preferred to y_l, the *Bradley–Terry model* posits

$$\mathbb{P}(y_w \succ y_l \mid x) = \sigma\big(r(x, y_w) - r(x, y_l)\big),$$

and r_ϕ is fitted by maximum likelihood. The policy then maximizes the reward while staying close to a reference model π_{ref}:

$$\max_{\pi} \ \mathbb{E}_{y\sim\pi(\cdot|x)}[r(x, y)] - \beta\, D_{\mathrm{KL}}\big(\pi(\cdot \mid x) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid x)\big). \tag{14.1}$$

Proposition 14.3. *The unique maximizer of* (14.1) *is* $\pi^*(y \mid x) = \frac{1}{Z(x)}\pi_{\mathrm{ref}}(y \mid x)e^{r(x,y)/\beta}$, *with* $Z(x) = \sum_y \pi_{\mathrm{ref}}(y \mid x)e^{r(x,y)/\beta}$.

Proof. For fixed x, the objective equals $\sum_y \pi(y)\big[r(y) - \beta \log \frac{\pi(y)}{\pi_{\mathrm{ref}}(y)}\big] = -\beta \sum_y \pi(y) \log \frac{\pi(y)}{\pi^*(y)} + \beta \log Z = \beta \log Z - \beta D_{\mathrm{KL}}(\pi \| \pi^*)$, maximized exactly at $\pi = \pi^*$ by Theorem 4.2. □

Solving for the reward, $r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\mathrm{ref}}(y|x)} + \beta \log Z(x)$; in the Bradley–Terry likelihood the intractable $Z(x)$ cancels. This gives *direct preference optimization* [31], which skips the reward model and the reinforcement learning loop and minimizes

$$-\mathbb{E} \log \sigma\Big(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)}\Big).$$

Constitutional methods replace some human comparisons by comparisons produced by a model following written principles.

Exercise 14.4. On a 4×4 gridworld with a terminal goal of reward 1, all other rewards 0 and $\gamma = 0.9$, compute V^* and an optimal policy by value iteration. How many iterations guarantee $\|V_k - V^*\|_\infty < 10^{-3}$ from $V_0 = 0$?

Exercise 14.5. For a softmax policy $\pi_\theta(a \mid s) \propto e^{\theta_{s,a}}$, compute $\nabla_\theta \log \pi_\theta$, and show that the variance-minimizing constant baseline for a one-step problem is $b = \mathbb{E}[R\|\nabla \log \pi\|^2]/\mathbb{E}[\|\nabla \log \pi\|^2]$.

15. Geometric deep learning

15.1. **Groups and equivariance.** A *group G acts linearly* on a vector space V through a homomorphism $\rho\colon G \to GL(V)$. A map $f\colon V \to W$ between two such spaces is *equivariant* if $f(\rho_V(g)v) = \rho_W(g)f(v)$ for all g, v, and *invariant* if $f(\rho_V(g)v) = f(v)$. Compositions of equivariant maps are equivariant, and an equivariant map followed by an invariant one is invariant. The *blueprint of geometric deep learning* [32] is to build networks as compositions of equivariant layers (linear equivariant maps and pointwise nonlinearities), local pooling, and a final invariant readout, for the symmetry group of the domain: translations for images (Proposition 10.1),

permutations for sets and graphs, rotations and translations for molecules. Symmetry reduces the number of parameters and guarantees generalization along group orbits.

15.2. **Sets.** The symmetric group S_n acts on $\mathbb{R}^n$ by permuting coordinates, $(P_\sigma x)_i = x_{\sigma^{-1}(i)}$.

Proposition 15.1. *Let $n \geq 2$. A linear map $L\colon \mathbb{R}^n \to \mathbb{R}^n$ is S_n-equivariant if and only if $L = aI + b\mathbf{1}\mathbf{1}^\top$ for some $a, b \in \mathbb{R}$. A linear functional is S_n-invariant if and only if it is $c\mathbf{1}^\top$.*

Proof. Equivariance means $P_\sigma L = LP_\sigma$ for all σ, i.e. $L_{\sigma(i)\sigma(j)} = L_{ij}$. The group acts transitively on the diagonal pairs (i,i) and on the off-diagonal pairs (i,j), $i \neq j$, so L has one constant value $a+b$ on the diagonal and another b off it. Conversely such L commute with all P_σ. The invariant case is the same argument for $\mathbf{1}^\top$-orbits. □

With feature channels, an equivariant layer on $X \in \mathbb{R}^{n\times d}$ is $X \mapsto XA + \frac{1}{n}\mathbf{1}\mathbf{1}^\top XB$: each element is transformed by itself and by the mean of the set. Stacking such layers with an invariant sum or mean at the end gives the Deep Sets architecture.

15.3. **Graphs and message passing.** A graph $G = (V, E)$ with n nodes, adjacency matrix A and node features $X \in \mathbb{R}^{n\times d}$ is acted on by S_n via $(A, X) \mapsto (PAP^\top, PX)$. A *message passing* layer updates every node by

$$\mathbf{h}'_v = \psi\Big(\mathbf{h}_v, \bigoplus_{u\in N(v)} \mu(\mathbf{h}_v, \mathbf{h}_u)\Big),$$

where $N(v)$ is the neighbourhood of v, μ and ψ are networks, and $\bigoplus$ is a permutation-invariant aggregation (sum, mean, maximum).

Proposition 15.2. *Message passing layers are permutation equivariant, and a sum or mean of the final node features is a permutation-invariant graph representation.*

Proof. Relabelling the nodes relabels neighbourhoods accordingly, and the aggregation does not depend on the order of the multiset $\{\mu(\mathbf{h}_v, \mathbf{h}_u) : u \in N(v)\}$. □

15.4. **Graph convolutional networks.** The GCN layer [33] is $H' = \phi(\hat{A}HW)$ with $\hat{A} = \tilde{D}^{-1/2}(A + I)\tilde{D}^{-1/2}$ and $\tilde{D}$ the degree matrix of $A + I$: each node averages itself and its neighbours with symmetric normalization, then applies a shared linear map. Its spectral meaning comes from the normalized Laplacian $\mathcal{L} = I - D^{-1/2}AD^{-1/2}$, the graph analogue of $-\Delta$.

Proposition 15.3. *For a graph without isolated vertices, $x^\top \mathcal{L} x = \sum_{\{i,j\}\in E}\big(\frac{x_i}{\sqrt{d_i}} - \frac{x_j}{\sqrt{d_j}}\big)^2$. Hence $\mathcal{L}$ is positive semidefinite with eigenvalues in $[0, 2]$, and $D^{1/2}\mathbf{1}$ is an eigenvector with eigenvalue 0.*

Proof. Expand the right side: each edge contributes $x_i^2/d_i + x_j^2/d_j - 2x_ix_j/\sqrt{d_id_j}$, and vertex i lies on d_i edges, giving $\|x\|^2 - x^\top D^{-1/2}AD^{-1/2}x$. For the upper bound, $(\alpha - \beta)^2 \leq 2\alpha^2 + 2\beta^2$ gives $x^\top \mathcal{L} x \leq 2\|x\|^2$. □

Eigenvectors of $\mathcal{L}$ with small eigenvalues vary slowly across edges, and play the role of low frequencies. The propagation matrix $D^{-1/2}AD^{-1/2} = I - \mathcal{L}$ multiplies the component along an eigenvector of eigenvalue λ by $1 - \lambda$: a GCN layer is a low-pass filter (the self-loops of $\tilde{A}$ shrink the spectrum away from -1). Stacking many layers smooths the features towards the bottom eigenvector, so that nodes become indistinguishable (*oversmoothing*).

15.5. **Expressive power: the Weisfeiler–Lehman test.** *Color refinement* (the 1-WL test) assigns every node an initial color $c^0(v)$ (its feature, or a constant) and updates $c^{k+1}(v) = \mathrm{hash}\big(c^k(v), \{\{c^k(u) : u \in N(v)\}\}\big)$ with an injective hash and multiset braces. Two graphs whose color histograms agree at every round are *WL-indistinguishable*.

Theorem 15.4 ([34, 35])**.** *If two graphs are WL-indistinguishable, every message passing network assigns them the same invariant representation. Conversely, there are message passing networks (with sum aggregation and injective multiset functions, such as GIN) that separate any two graphs that WL distinguishes.*

Proof of the first part. By induction on k, the features after k layers are a function of the WL colors after k rounds: if $c^k(v) = c^k(w)$, then v, w have equal features and equal multisets of neighbour colors, hence equal multisets of neighbour features, hence equal updated features. Equal histograms of colors then give equal sums of features. □

Example 15.5. The 6-cycle C_6 and the disjoint union of two triangles are both 2-regular on 6 vertices; with constant initial colors, refinement never splits the nodes, so no message passing network can tell a connected graph from a disconnected one in this case, nor count triangles.

15.6. **Euclidean symmetry in three dimensions.** A molecule is a set of atoms with types and positions $\mathbf{r}_i \in \mathbb{R}^3$. Its energy is invariant, and the forces on its atoms are equivariant, under the Euclidean group $E(3)$ of rotations, reflections and translations, as well as under permutations of identical atoms. Functions of the pairwise distances $\|\mathbf{r}_i - \mathbf{r}_j\|$ are automatically $E(3)$-invariant; conversely, by a classical theorem of invariant theory, every $O(3)$-invariant polynomial of the vectors $\mathbf{r}_i - \mathbf{r}_j$ is a polynomial in their inner products. Equivariant networks either pass messages built from distances (and angles), or carry vector and tensor features that transform under representations of $SO(3)$. Equivariant architectures of this kind underlie protein structure prediction and machine-learned interatomic potentials.

Exercise 15.6. Classify the linear S_n-equivariant maps $\mathbb{R}^{n\times n} \to \mathbb{R}^{n\times n}$ (acting by $X \mapsto PXP^\top$) and show that they form a space of dimension 15 for $n \geq 4$.

Exercise 15.7. Run color refinement on the two graphs with vertex set $\{1, \dots, 8\}$ given by the cube graph Q_3 and by the cycle C_8 with the four long diagonals $\{i, i+4\}$ added. Are they distinguished? Are they isomorphic?

Exercise 15.8. Show that the function $f(\mathbf{r}_1, \dots, \mathbf{r}_N) = \sum_{i<j} \varphi(\|\mathbf{r}_i - \mathbf{r}_j\|)$ is $E(3)$- and permutation-invariant, and that $-\nabla_{\mathbf{r}_i} f$ is equivariant: rotating the configuration rotates the forces.

References

[1] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016.
[2] M. P. Deisenroth, A. A. Faisal, C. S. Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020.
[3] C. C. Aggarwal, *Neural Networks and Deep Learning*, 2nd ed., Springer, 2023.
[4] L. Berlyand, P.-E. Jabin, *Mathematics of Deep Learning: An Introduction*, De Gruyter, 2023.
[5] C. Eckart, G. Young, The approximation of one matrix by another of lower rank, *Psychometrika* **1** (1936), 211–218.
[6] M. Belkin, D. Hsu, S. Ma, S. Mandal, Reconciling modern machine-learning practice and the classical bias–variance trade-off, *Proc. Natl. Acad. Sci. USA* **116** (2019), 15849–15854.
[7] D. H. Wolpert, The lack of a priori distinctions between learning algorithms, *Neural Comput.* **8** (1996), 1341–1390.
[8] H. Robbins, S. Monro, A stochastic approximation method, *Ann. Math. Statist.* **22** (1951), 400–407.
[9] D. P. Kingma, J. Ba, Adam: a method for stochastic optimization, ICLR 2015.
[10] C. Cortes, V. Vapnik, Support-vector networks, *Mach. Learn.* **20** (1995), 273–297.
[11] N. Aronszajn, Theory of reproducing kernels, *Trans. Amer. Math. Soc.* **68** (1950), 337–404.

[12] M. Leshno, V. Ya. Lin, A. Pinkus, S. Schocken, Multilayer feedforward networks with a nonpolynomial activation function can approximate any function, *Neural Netw.* **6** (1993), 861–867.
[13] G. Cybenko, Approximation by superpositions of a sigmoidal function, *Math. Control Signals Systems* **2** (1989), 303–314.
[14] W. Baur, V. Strassen, The complexity of partial derivatives, *Theoret. Comput. Sci.* **22** (1983), 317–330.
[15] K. He, X. Zhang, S. Ren, J. Sun, Delving deep into rectifiers, ICCV 2015.
[16] S. Hochreiter, J. Schmidhuber, Long short-term memory, *Neural Comput.* **9** (1997), 1735–1780.
[17] A. Vaswani et al., Attention is all you need, NeurIPS 2017.
[18] P. Baldi, K. Hornik, Neural networks and principal component analysis: learning from examples without local minima, *Neural Netw.* **2** (1989), 53–58.
[19] H. Li et al., A survey on large language model acceleration based on KV cache management, arXiv:2412.19442.
[20] A. Fichtl et al., The end of Transformers? On challenging attention and the rise of sub-quadratic architectures, arXiv:2510.05364.
[21] A. Gu, T. Dao, Mamba: linear-time sequence modeling with selective state spaces, arXiv:2312.00752.
[22] M. E. Tipping, C. M. Bishop, Probabilistic principal component analysis, *J. R. Stat. Soc. B* **61** (1999), 611–622.
[23] D. P. Kingma, M. Welling, Auto-encoding variational Bayes, arXiv:1312.6114.
[24] D. J. Rezende, S. Mohamed, Variational inference with normalizing flows, ICML 2015.
[25] J. Ho, A. Jain, P. Abbeel, Denoising diffusion probabilistic models, NeurIPS 2020.
[26] P. Vincent, A connection between score matching and denoising autoencoders, *Neural Comput.* **23** (2011), 1661–1674.
[27] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, B. Poole, Score-based generative modeling through stochastic differential equations, ICLR 2021.
[28] B. D. O. Anderson, Reverse-time diffusion equation models, *Stochastic Process. Appl.* **12** (1982), 313–326.
[29] R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.
[30] R. J. Williams, Simple statistical gradient-following algorithms for connectionist reinforcement learning, *Mach. Learn.* **8** (1992), 229–256.
[31] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, C. Finn, Direct preference optimization, NeurIPS 2023.
[32] M. M. Bronstein, J. Bruna, T. Cohen, P. Veličković, Geometric deep learning: grids, groups, graphs, geodesics, and gauges, arXiv:2104.13478.
[33] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, ICLR 2017.
[34] K. Xu, W. Hu, J. Leskovec, S. Jegelka, How powerful are graph neural networks?, ICLR 2019.
[35] C. Morris et al., Weisfeiler and Leman go neural: higher-order graph neural networks, AAAI 2019.

Facultad de Empresariales, Universidad Panamericana, Ciudad de México, México
Email address: jcgalindo@up.edu.mx